NetApp上的BeeGFS与E系列存储

BeeGFS on NetApp with E-Series Storage

NetApp
August 01, 2022

This PDF was generated from https://docs.netapp.com/zh-cn/beegfs/index.html on August 01, 2022.
Always check docs.netapp.com for the latest.

目录

NetApp上的BeeGFS与E系列存储 1
入门 2
此站点包含的内容 2
术语和概念 2
设计代数 3
解决方案概述 4
架构概述 5
技术要求 7
查看解决方案 设计 12
设计概述 12
硬件配置 12
软件配置 13
设计验证 16
规模估算准则 19
性能调整 20
部署解决方案 23
部署概述 23
了解Ansible清单 24
查看最佳实践 26
部署硬件 28
部署软件 30
扩展到五个组件以上 64
建议的存储池过度配置百分比 64
法律声明 66
版权 66
商标 66
专利 66
隐私政策 66
开放源代码 66

NetApp上的BeeGFS与E系列存储

入门

此站点包含的内容

此站点提供了有关NetApp上BeeGFS的第二代NetApp验证架构(NVA)的特定信息。

此站点中的信息包括：

- 用于确定支持架构所需的特定设备、布线和配置的架构设计注意事项。有关详细信息，请参见 "设计概述"。
- 使用连接到块存储系统(块节点)的经过验证的BeeGFS文件服务器(文件节点)的部署准则。有关详细信息，请参见 "部署概述"。

此站点不包含第一代解决方案 的设计和部署信息。有关上一代产品的完整内容、请参见 "采用NVIDIA DGX A100系统和BeeGFS的NetApp EF系列AI"。

术语和概念

以下术语和概念适用于NetApp解决方案 上的BeeGFS。

期限	**Description**
AI	人工智能。
可逆清单	包含用于描述所需BeeGFS HA集群的YAML文件的目录结构。
BMC	基板管理控制器。有时称为服务处理器。
块节点	存储系统。
客户端	服务器节点。
深度	深度学习。
文件节点	BeeGFS文件服务器。
HA	高可用性。
HIC	主机接口卡。
HPC	高性能计算。

期限	Description
HPC模式的工作负载	HPC模式的工作负载通常具有多个计算节点或GPU、这些节点或GPU都需要并行访问同一数据集、以便执行分布式计算或培训作业。这些数据集通常由大型文件组成、这些文件应在多个物理存储节点之间进行条带化、以消除传统硬件瓶颈、避免同时访问单个文件。
毫升	机器学习。
NLP	自然语言处理。
NLU	自然语言理解。
NVA	NetApp经验证的架构(NVA)计划为特定工作负载和使用情形提供参考配置和规模估算指导。这些解决方案经过全面测试、旨在最大程度地降低部署风险并加快上市速度。

设计代数

NetApp解决方案 上的BeeGFS目前正处于第二代设计阶段。

第一代和第二代均采用一种基础架构、其中整合了BeeGFS文件系统和NVMe EF600存储系统。但是、第二代基于第一代产品构建、其中包括以下附加优势：

- 将性能和容量提高一倍、同时仅添加2U机架空间
- 高可用性(High Availability、HA)基于共享磁盘的两层硬件设计
- NVIDIA DGX A100 SuperPOD的外部资格认证

第二代设计

基于NetApp的第二代BeeGFS经过优化、可满足高性能计算(HPC)和HPC模式机器学习(ML)、深度学习(DL)以及类似人工智能(AI)技术等苛刻工作负载的性能要求。通过整合共享磁盘高可用性(HA)架构、NetApp解决方案 上的BeeGFS还可以满足企业和其他组织在寻找可扩展的存储以适应其工作负载和使用情形时无法承受停机或数据丢失的需求。此解决方案 不仅已通过NetApp的验证、还通过了外部认证、作为NVIDIA DGX SuperPOD的存储选项。

第一代设计

第一代基于NetApp的BeeGFS专为使用NetApp EF600 NVMe存储系统、BeeGFS并行文件系统、NVIDIA DGX®A100系统和NVIDIA®Mellanox®Quantum®QM8700 200 Gbps IB交换机的机器学习(ML)和人工智能(AI)工作负载而设计。此设计还为存储和计算集群互连网络结构提供了200 Gbps InfiniBand (IB)、可为高性能工作负载提供完全基于IB的架构。

有关第一代产品的详细信息、请参见 "采用NVIDIA DGX A100系统和BeeGFS的NetApp EF系列AI"。

解决方案概述

NetApp解决方案 上的BeeGFS将BeeGFS并行文件系统与NetApp EF600存储系统相结合、打造了一个可靠、可扩展且经济高效的基础架构、可满足苛刻的工作负载要求。

这种设计利用了最新企业级服务器和存储硬件以及网络速度所提供的性能密度、 要求文件节点采用双AMD EPYC 7003 "米兰"处理器并支持使用200 GB (HDR) InfiniBand直接连接的PCIe 4.0、以块节点使用NVMe/IB协议提供端到端NVMe和NVMeOF。

NVA计划

NetApp解决方案 上的BeeGFS是NetApp验证架构(NVA)计划的一部分、该计划为客户提供针对特定工作负载和使用情形的参考配置和规模估算指导。NVA解决方案经过全面测试和设计、可最大限度地降低部署风险并加快上市速度。

用例

以下使用情形适用于NetApp解决方案 上的BeeGFS：

- 人工智能(AI)、包括机器学习(ML)、深度学习(DL)、大规模自然语言处理(NLP)和自然语言理解(NLF)。有关详细信息，请参见 "BeeGFS for AI：事实与虚构"。
- 高性能计算(HPC)、包括通过MPI (消息传递接口)和其他分布式计算技术加速的应用程序。有关详细信息，请参见 "BeeGFS为何超越HPC"。
- 应用程序工作负载的特征为：
  - 读取或写入大于1 GB的文件
  - 多个客户端(10、100和1000)读取或写入同一文件
- 多TB或多PB数据集。
- 需要一个存储命名空间的环境、可以优化大小文件的组合。

优势

在NetApp上使用BeeGFS的主要优势包括：

- 提供经验证的硬件设计、可实现硬件和软件组件的全面集成、以确保可预测的性能和可靠性。
- 使用Ansible进行部署和管理、以实现规模化的精简性和一致性。
- 使用E系列性能分析器和BeeGFS插件提供监控和可观察性。有关详细信息，请参见 "介绍用于监控NetApp E系列解决方案的框架"。
- 高可用性采用共享磁盘架构、可提供数据持久性和可用性。
- 支持使用容器和Kubernetes进行现代工作负载管理和流程编排。有关详细信息，请参见 "Kubernetes认识BeeGFS：一个适应未来需求的投资故事"。

HA架构

NetApp上的BeeGFS通过创建与NetApp硬件完全集成的解决方案 来扩展BeeGFS企业版的功能、从而实现共享磁盘高可用性(HA)架构。

> 虽然BeeGFS社区版可以免费使用、但企业版要求从NetApp等合作伙伴购买专业支持订阅合同。企业版支持使用多项附加功能、包括故障恢复能力、配额强制实施和存储池。

下图对无共享和共享磁盘HA架构进行了比较。

[]

有关详细信息，请参见 "宣布NetApp支持的BeeGFS高可用性"。

Ansible

NetApp上的BeeGFS是使用Ansible自动化交付和部署的、Ansible自动化托管在GitHub和Ansible GALAXY上(BeeGFS集合可从获取 "Ansible Galax河" 和 "NetApp的E系列GitHub"）。虽然Ansible主要是使用用于组装BeeGFS组件的硬件进行测试的、但您可以将其配置为使用受支持的Linux版本在几乎任何基于x86的服务器上运行。

有关详细信息，请参见 "使用E系列存储部署BeeGFS"。

架构概述

NetApp解决方案 上的BeeGFS包括用于确定支持经验证的工作负载所需的特定设备、布线和配置的架构设计注意事项。

构建块架构

根据存储要求、可以使用不同的方式部署和扩展BeeGFS文件系统。例如、以大量小文件为主要特征的使用情形将受益于额外的元数据性能和容量、而大文件较少的使用情形则可能有利于提高实际文件内容的存储容量和性能。这些多种注意事项会影响并行文件系统部署的不同维度、从而增加了文件系统设计和部署的复杂性。

为了应对这些挑战、NetApp设计了一个标准的构建块架构、用于横向扩展其中的每个方面。通常、BeeGFS组件部署在以下三种配置文件之一中：

- 一个基础组件、包括BeeGFS管理、元数据和存储服务
- BeeGFS元数据加上存储组件
- 仅供BeeGFS存储使用的组件

这三个选项之间的唯一硬件更改是使用较小的驱动器来获取BeeGFS元数据。否则、所有配置更改将通过软件应用。借助Ansible作为部署引擎、为特定组件设置所需的配置文件可以使配置任务变得简单明了。

有关更多详细信息、请参见 [Verified hardware design]。

文件系统服务

BeeGFS文件系统包括以下主要服务：

- *管理服务。*注册并监控所有其他服务。
- *存储服务。*存储称为数据区块文件的分布式用户文件内容。
- *元数据服务。*跟踪文件系统布局、目录、文件属性等。

- *客户端服务。*挂载文件系统以访问存储的数据。

下图显示了NetApp E系列系统中使用的BeeGFS解决方案 组件和关系。

[]

作为一个并行文件系统、BeeGFS会将其文件条带化到多个服务器节点上、以最大限度地提高读/写性能和可扩展性。服务器节点协同工作、可提供一个文件系统、其他服务器节点(通常称为_clients_）可以同时挂载和访问该文件系统。这些客户端可以查看和使用分布式文件系统、类似于NTFS、XFS或ext4等本地文件系统。

这四项主要服务在广泛支持的Linux分发版上运行、并通过任何支持TCP/IP或RDMA的网络进行通信、包括InfiniBand (IB)、OMNI-Path (OPA)和基于融合以太网的RDMA (RoCE)。BeeGFS服务器服务(管理、存储和元数据)是用户空间守护进程、而客户端是原生 内核模块(无修补)。所有组件均可在不重新启动的情况下安装或更新、您可以在同一节点上运行任何服务组合。

已验证节点

NetApp解决方案 上的BeeGFS包括以下经验证的节点：NetApp EF600存储系统(块节点)和联想ThinkSystem SR665服务器(文件节点)。

块节点：**EF600**存储系统

NetApp EF600全闪存阵列可提供一致、近乎实时的数据访问、同时支持任意数量的工作负载。为了快速、持续地向AI和HPC应用程序馈送数据、EF600存储系统在一个机箱中提供高达200万个缓存读取IOPS、低于100微秒的响应时间以及42 GBps顺序读取带宽。

文件节点：**Lenovo ThinkSystem SR665**服务器

SR665是一款采用PCIe 4.0的双插槽2U服务器。如果配置为满足此解决方案 的要求、则它可以提供充足的性能、以便在与直连E系列节点提供的吞吐量和IOPS相当的配置中运行BeeGFS文件服务。

有关联想SR665的详细信息、请参见 "联想的网站"。

经验证的硬件设计

该解决方案的组件(如下图所示)使用两个双插槽PCIe 4.0服务器作为BeeGFS文件层、并使用两个EF600存储系统作为块层。

[]

> 由于每个组件都包含两个BeeGFS文件节点、因此要在故障转移集群中建立仲裁、至少需要两个组件。虽然您可以配置双节点集群、但此配置具有一些限制、可能会阻止成功进行故障转移。如果您需要双节点集群、则可以将第三个设备作为Tiebreaker (但是、此站点不会介绍此设计)。

每个组件都通过两层硬件设计提供高可用性、这种设计可分隔文件层和块层的故障域。每个层都可以独立进行故障转移、从而提高故障恢复能力并降低级联故障的风险。将HDR InfiniBand与NVMeOF结合使用可在文件和块节点之间提供高吞吐量和最低延迟、并提供完全冗余和足够的链路超额预订、以避免分散设计成为瓶颈、即使系统部分降级也是如此。

NetApp解决方案 上的BeeGFS可跨部署中的所有组件运行。部署的第一个组件必须运行BeeGFS管理、元数据和存储服务(称为基础组件)。所有后续组件均通过软件配置为运行BeeGFS元数据和存储服务、或者仅运行存储服务。由于每个组件都具有不同的配置配置文件、因此可以使用相同的底层硬件平台和组件设计扩展文件系统元

数据或存储容量和性能。

一个独立的Linux HA集群最多可组合五个组件、从而确保每个集群资源管理器(Pacemaker)有合理数量的资源、并降低保持集群成员同步所需的消息传送开销(Corosync)。建议每个集群至少有两个组件、以便有足够的成员建立仲裁。将这些独立BeeGFS HA集群中的一个或多个组合在一起、以创建一个BeeGFS文件系统(如下图所示)、此文件系统可作为一个存储命名空间供客户端访问。

[]

尽管每个机架的组件数量最终取决于给定站点的电源和散热要求、 解决方案 经过专门设计、可在一个42U机架中部署多达五个组件、同时仍可为用于存储/数据网络的两个1U InfiniBand交换机提供空间。每个构建块需要八个IB端口(每个交换机四个用于实现冗余)、因此五个构建块会在一个40端口HDR InfiniBand交换机(如NVIDIA QM8700)上保留一半的端口、以便实施ft-tree或类似的非阻塞拓扑。此配置可确保存储或计算/GPU机架的数量可以纵向扩展、而不会出现网络瓶颈。或者、可以根据存储网络结构供应商的建议使用超额预订的存储网络结构。

下图显示了一个80节点的ft-tree拓扑。

[]

通过使用Ansible作为部署引擎在NetApp上部署BeeGFS、管理员可以使用现代基础架构作为代码实践来维护整个环境。这样可以极大地简化原本复杂的系统、使管理员可以在一个位置定义和调整所有配置、然后确保无论环境扩展多大、都能始终如一地应用。可从获取BeeGFS集合 "Ansible Galax河" 和 "NetApp的E系列GitHub"。

技术要求

要在NetApp解决方案 上实施BeeGFS、请确保您的环境满足技术要求。

硬件要求

下表列出了在NetApp解决方案 上实施BeeGFS的单个第二代组件设计所需的硬件组件。

> 在任何特定解决方案 实施中使用的硬件组件可能会因客户要求而异。

计数	硬件组件	要求
2.	BeeGFS文件节点。	每个文件节点都应满足或超出以下配置、以实现预期性能。 处理器： • 2个AMD EPYC 7343 16C 3.2 GHz。 • 配置为两个NUMA分区。 内存： • 256 GB。 • 16个16 GB TruDDR4 3200MHz (2Rx8 1.2V) RDIMM-A (首选较小的DIMM、而不是较大的DIMM)。 • 填充以最大程度地提高内存带宽。 • PCIe扩展：四个PCE第4代x16插槽：* • 每个NUMA分区有两个插槽。 • 每个插槽都应为Mellanox MCX653106A-HDAT适配器提供足够的电源/散热。 其他： • 在RAID 1中为操作系统配置两个1 TB 7.2K SATA驱动器(或类似驱动器)。 • 10GbE OCP 3.0适配器(或类似适配器)、用于带内操作系统管理。 • 采用Redfish API的1GbE BMC、用于带外服务器管理。 • 双热插拔电源和高性能风扇。 • 如果需要连接到存储InfiniBand交换机、则必须支持Mellanox光纤InfiniBand缆线。 联想**SR665**： • 自定义NetApp型号包括支持双端口Mellanox ConnectX-6适配器所需的XiclityL控制器固件版本。有关订购详细信息、请联系NetApp。
8.	Mellanox ConnectX-6 HCA (适用于文件节点)。	• MCX653106A-HDAT主机通道适配器(HDR IB 200GB、双端口QSFP56、PCIe4.0 x16)。
8.	100万根HDR InfiniBand缆线(用于文件/块节点直接连接)。	• MCP1650-H001E30 (1米Mellanox无源铜缆、IB HDR、高达200 Gbps、QSFP56、30AWM)。 如果需要、可以调整此长度、以考虑文件节点与块节点之间的距离较长。

计数	硬件组件	要求
8.	HDR InfiniBand缆线(用于文件节点/存储交换机连接)	需要长度适当的InfiniBand HDR缆线(QSFP56收发器)、才能将文件节点连接到存储叶交换机。可能的选项包括： • MCP1650-H002E26 (2米Mellanox无源铜缆、IB HDR、高达200 GB/秒、QSFP56、30AWM)。 • MFS1S00-H003E (3米Mellanox有源光缆、IB HDR、高达200 GB/秒、QSFP56)。
2.	E系列块节点	两个EF600控制器配置如下： • 内存：256 GB (每个控制器128 GB)。 • 适配器：2端口200 GB/HDR (NVMe/IB)。 • Drives：已配置为与所需容量匹配。

软件要求

为了获得可预测的性能和可靠性、NetApp解决方案 上的BeeGFS版本会使用实施解决方案 所需的特定版本软件组件进行测试。

软件部署要求

下表列出了在基于Ansible的BeeGFS部署中自动部署的软件要求。

软件	**version**
BeeGFS	7.2.6
核心同步	3.1.5-1
起搏器	2.1.0-8.
opensm	openssm-5.9.0 (从mlnx_OFED 5.4.1.0.3.0开始) ⓘ 只有直接连接才需要启用虚拟化。

Ansible控制节点要求

NetApp解决方案 上的BeeGFS可从Ansible控制节点进行部署和管理。有关详细信息，请参见 "Ansible文档"。

下表中列出的软件要求特定于下面列出的NetApp BeeGFS Ansible集合版本。

软件	**version**
Ansible	2.11通过pip安装时：Ansible 4.7.0和Ansible核心< 2.12、>=2.11.6
Python	3.9.
其他Python软件包	加密-35.0.0、netaddr-0.8.0

软件	**version**
BeeGFS Ansible资料集	3.0.0

文件节点要求

软件	**version**
RedHat Enterprise Linux	具有高可用性的RedHat 8.4 Server物理服务器(双插槽)。 文件节点需要有效的RedHat Enterprise Linux Server订阅和Red Hat Enterprise Linux高可用性附加软件。
Linux内核	4.18.0-305.25.1.el8_4.x86_64
InfiniBand / RDMA驱动程序	收件箱
ConnectX-6 HCA固件	固件：20.31.1014
PXE：3.6.0403	UEFI：14.24.0013

EF600块节点要求

软件	**version**
SANtricity 操作系统	11.70.2
NVSRAM	N6000-872834-D06.DLP
驱动器固件	适用于所用驱动器型号的最新版本。

其他要求

下表中列出的设备用于验证、但可以根据需要使用适当的替代设备。通常、NetApp建议运行最新版本的软件、以避免意外问题。

硬件组件	已安装软件
• 2个Mellanox MQM8700 200 GB InfiniBand交换机	• 固件3.9.2110
• 1个可变控制节点(虚拟化)：* • 处理器：Intel ® Xeon ® Gold 6146 CPU @ 3.20GHz • 内存：8 GB • 本地存储：24 GB	• CentOS Linux 8.4.2105 • 内核4.18.0-305.3.1.el8.x86_64 已安装的Ansible和Python版本与上表中的版本匹配。

硬件组件	已安装软件
• 10个BeeGFS客户端(CPU节点)：* • 处理器：1个3.0 GHz AMD EPYC 7302 16核CPU • 内存：128 GB • 网络：2个Mellanox MCX653106A-HDAT (每个适配器连接一个端口)。	• Ubuntu 20.04 • 内核：5.4.0.100-generic • InfiniBand驱动程序：Mellanox OFED 5.3-1.0.3.0
• 1个BeeGFS客户端(GPU节点)：* • 处理器：2个2.5 GHz AMD EPYC 7742 64核CPU • 内存：1 TB • 网络：2个Mellanox MCX653106A-HDAT (每个适配器连接一个端口)。 此系统基于NVIDIA HGX A100平台、并包括四个A100 GPU。	• Ubuntu 20.04 • 内核：5.4.0.100-generic • InfiniBand驱动程序：Mellanox OFED 5.3-1.0.3.0

查看解决方案 设计

设计概述

要支持NetApp解决方案 上的BeeGFS、需要使用特定的设备、布线和配置、该BeeGFS并行文件系统与NetApp EF600存储系统相结合。

了解更多信息。

- "硬件配置"
- "软件配置"
- "设计验证"
- "规模估算准则"
- "性能调整"

硬件配置

NetApp上BeeGFS的硬件配置包括文件节点和网络布线。

文件节点配置

文件节点具有两个CPU插槽、这些插槽配置为单独的NUMA分区、其中包括对相同数量的PCIe插槽和内存的本地访问。

InfiniBand适配器必须填充到相应的PCI提升板或插槽中、以便在可用的PCIe通道和内存通道上平衡工作负载。您可以通过将单个BeeGFS服务的工作完全隔离到特定NUMA节点来平衡工作负载。目标是从每个文件节点获得类似的性能、就像两个独立的单插槽服务器一样。

下图显示了文件节点NUMA配置。

[]

BeeGFS进程固定到特定的NUMA区域、以确保使用的接口位于同一区域。此配置可避免通过插槽间连接进行远程访问。插槽间连接有时称为QPI或GMI2链路；即使在现代处理器架构中、也可能会在使用诸如HDR InfiniBand等高速网络时成为瓶颈。

网络布线配置

在一个构建块中、每个文件节点使用总共四个冗余InfiniBand连接连接连接到两个块节点。此外、每个文件节点还具有四个到InfiniBand存储网络的冗余连接。

在下图中、请注意：

- 绿色中所示的所有文件节点端口均用于连接到存储网络结构；所有其他文件节点端口均直接连接到块节点。
- 特定NUMA区域中的两个InfiniBand端口连接到同一块节点的A和B控制器。
- NUMA节点0中的端口始终连接到第一个块节点。

- NUMA节点1中的端口连接到第二个块节点。

[]

> 对于具有冗余交换机的存储网络、浅绿色端口应连接到一个交换机、深绿色端口应连接到另一个交换机。

图中所示的布线配置允许每个BeeGFS服务：

- 无论运行BeeGFS服务的文件节点是什么、都可以在同一NUMA区域中运行。
- 无论故障发生在何处、都要有通向前端存储网络和后端块节点的二级最佳路径。
- 如果块节点中的文件节点或控制器需要维护、则最大限度地降低性能影响。

要利用完整的PCIe双向带宽、请确保每个InfiniBand适配器上的一个端口连接到存储网络结构、另一个端口连接到块节点。理论上、一个HDR InfiniBand端口的最大速度为25 GBps (不考虑信号传输和其他开销)。PCIe 4.0 x16插槽的最大单向带宽为32 Gbps、如果实施的文件节点采用双端口InfiniBand适配器、理论上可处理50 GB的带宽、则可能会造成瓶颈。

下图显示了用于利用完整PCIe双向带宽的布线设计。

[]

对于每个BeeGFS服务、请使用同一适配器将用于客户端流量的首选端口连接到作为该服务卷的主所有者的块节点控制器的路径。有关详细信息，请参见 "软件配置"。

软件配置

NetApp上的BeeGFS软件配置包括BeeGFS网络组件、EF600块节点、BeeGFS文件节点、资源组和BeeGFS服务。

BeeGFS网络配置

BeeGFS网络配置由以下组件组成。

- *浮动IP*浮动IP是一种虚拟IP地址、可以动态路由到同一网络中的任何服务器。多个服务器可以拥有相同的浮动IP地址、但在任何给定时间、它只能在一个服务器上处于活动状态。

  每个BeeGFS服务器服务都有自己的IP地址、可以根据BeeGFS服务器服务的运行位置在文件节点之间移动。此浮动IP配置允许每个服务单独故障转移到另一个文件节点。客户端只需知道特定BeeGFS服务的IP地址即可；它不需要知道哪个文件节点当前正在运行该服务。

- * BeeGFS服务器多主机配置*为了提高解决方案 的密度、每个文件节点都有多个存储接口、其IP配置在同一IP子网中。

  为了确保此配置在Linux网络堆栈中正常运行、需要进行其他配置、因为默认情况下、如果一个接口的IP位于同一子网中、则可以在其他接口上对对对对请求做出响应。除了其他缺点之外、此默认行为还会使您无法正确建立或维护RDMA连接。

  基于Ansible的部署可处理反向路径(RP)和地址解析协议(ARP)行为的收紧、并确保浮动IP何时启动和停止；系统会动态创建相应的IP路由和规则、以使多宿主网络配置正常运行。

- * BeeGFS客户端多导轨配置*_Multi-raid_是指应用程序能够使用多个独立的网络"导轨"来提高性能。

  虽然BeeGFS可以使用RDMA进行连接、但BeeGFS使用IPoIB简化发现和建立RDMA连接的过程。要允许BeeGFS客户端使用多个InfiniBand接口、您可以为每个客户端配置位于不同子网中的IP地址、然后为每个子网中一半的BeeGFS服务器服务配置首选接口。

  在下图中、以淡绿色突出显示的接口位于一个IP子网中(例如、`100.127.0.0/16`)、而深绿色接口位于另一个子网中(例如、`100.128.0.0/16`)。

  下图显示了在多个BeeGFS客户端接口之间平衡流量的情况。

  []

  由于BeeGFS中的每个文件通常在多个存储服务之间进行条带化、因此多导轨配置可以使客户端实现比单个InfiniBand端口更大的吞吐量。例如、以下代码示例显示了一个通用文件条带化配置、该配置允许客户端在两个接口之间平衡流量：

  ```
  root@ictad21h01:/mnt/beegfs# beegfs-ctl --getentryinfo myfile
  Entry type: file
  EntryID: 11D-624759A9-65
  Metadata node: meta_01_tgt_0101 [ID: 101]
  Stripe pattern details:
  + Type: RAID0
  + Chunksize: 1M
  + Number of storage targets: desired: 4; actual: 4
  + Storage targets:
    + 101 @ stor_01_tgt_0101 [ID: 101]
    + 102 @ stor_01_tgt_0101 [ID: 101]
    + 201 @ stor_02_tgt_0201 [ID: 201]
    + 202 @ stor_02_tgt_0201 [ID: 201]
  ```

  使用两个IPoIB子网是一个逻辑上的区别。如果需要、您可以使用一个物理InfiniBand子网(存储网络)。

  > BeeGFS 7.3.0增加了多轨支持、允许在一个IPoIB子网中使用多个IB接口。NetApp解决方案上的BeeGFS设计是在BeeGFS 7.3.0全面推出之前开发的、因此展示了如何使用两个IP子网在BeeGFS客户端上使用两个IB接口。多IP子网方法的一个优势是无需在BeeGFS客户端节点上配置多主节点(有关详细信息、请参见 "BeeGFS RDMA支持"）。

EF600块节点配置

块节点由两个主动/主动RAID控制器组成、这些控制器可共享访问同一组驱动器。通常、每个控制器拥有系统上配置的一半卷、但可以根据需要接管另一个控制器。

文件节点上的多路径软件可确定每个卷的活动和优化路径、并在发生缆线、适配器或控制器故障时自动移至备用路径。

下图显示了EF600块节点中的控制器布局。

[]

为了便于使用共享磁盘HA解决方案 、卷会映射到两个文件节点、以便它们可以根据需要相互接管。下图显示了如何配置BeeGFS服务和首选卷所有权以实现最高性能的示例。每个BeeGFS服务左侧的接口指示客户端和其他服务用来与其联系的首选接口。

[]

在上一示例中、客户端和服务器服务更愿意使用接口i1b与存储服务1进行通信。存储服务1使用接口i1a作为与其第一个块节点的控制器A上的卷(storage_tgt_101、102)进行通信的首选路径。此配置可利用InfiniBand适配器可用的全双向PCIe带宽、并通过双端口HDR InfiniBand适配器获得比PCIe 4.0更高的性能。

BeeGFS文件节点配置

BeeGFS文件节点配置到高可用性(HA)集群中、以便于在多个文件节点之间对BeeGFS服务进行故障转移。

HA集群设计基于两个广泛使用的Linux HA项目：用于集群成员资格的corosync和用于集群资源管理的Pacemaker。有关详细信息，请参见 "针对高可用性附加组件的Red Hat培训"。

NetApp编写并扩展了多个开放式集群框架(Open Cluster Framework、OCF)资源代理、使集群能够智能启动和监控BeeGFS资源。

BeeGFS HA集群

通常、在启动BeeGFS服务(无论是否具有HA)时、必须具备一些资源：

- 可访问服务的IP地址、通常由Network Manager配置。
- 用作BeeGFS数据存储目标的底层文件系统。

  这些参数通常在`/etc/fstab`中定义、并由systemd挂载。

- 一种系统服务、负责在其他资源准备就绪时启动BeeGFS进程。

  如果没有其他软件、这些资源只会在一个文件节点上启动。因此、如果文件节点脱机、则无法访问BeeGFS文件系统的一部分。

由于多个节点可以启动每个BeeGFS服务、因此Pacemaker必须确保每个服务和相关资源一次仅在一个节点上运行。例如、如果两个节点尝试启动同一BeeGFS服务、则在它们都尝试写入底层目标上的相同文件时、存在数据损坏的风险。为了避免这种情况、Pacemaker会依靠Corosync可靠地在所有节点之间保持整个集群的状态同步并建立仲裁。

如果集群发生故障、Pacemaker会做出反应、并在另一节点上重新启动BeeGFS资源。在某些情况下、Pacemaker可能无法与原始故障节点通信以确认资源已停止。要在其他位置重新启动BeeGFS资源之前验证节点是否已关闭、Pacemaker最好断开故障节点的电源。

有许多开源隔离代理可用于使Pacemaker使用配电单元(PDU)或使用服务器基板管理控制器(BMC)和Redfo* 等API来隔离节点。

当BeeGFS在HA集群中运行时、所有BeeGFS服务和底层资源都由资源组中的Pacemaker管理。每个BeeGFS服务及其所依赖的资源都会配置到一个资源组中、以确保资源按正确顺序启动和停止、并放置在同一节点上。

对于每个BeeGFS资源组、Pacemaker都会运行一个自定义BeeGFS监控资源、该资源负责检测故障情况、并在

特定节点上无法再访问BeeGFS服务时智能地触发故障转移。

下图显示了由Pacemaker控制的BeeGFS服务和依赖项。

[]

> 为了在同一节点上启动多个相同类型的BeeGFS服务、我们将Pacemaker配置为使用多模式配置方法启动BeeGFS服务。有关详细信息，请参见 "有关多模式的BeeGFS文档"。

由于BeeGFS服务必须能够在多个节点上启动、因此每个服务(通常位于`/etc/beegfs`)的配置文件存储在用作该服务的BeeGFS目标的E系列卷之一上。这样、可能需要运行此服务的所有节点都可以访问此配置以及特定BeeGFS服务的数据。

```
# tree stor_01_tgt_0101/ -L 2
stor_01_tgt_0101/
├── data
│   ├── benchmark
│   ├── buddymir
│   ├── chunks
│   ├── format.conf
│   ├── lock.pid
│   ├── nodeID
│   ├── nodeNumID
│   ├── originalNodeID
│   ├── targetID
│   └── targetNumID
└── storage_config
    ├── beegfs-storage.conf
    ├── connInterfacesFile.conf
    └── connNetFilterFile.conf
```

设计验证

NetApp解决方案 上的BeeGFS的第二代设计已使用三个组件配置文件进行了验证。

配置配置文件包括以下内容：

- 一个基础组件、包括BeeGFS管理、元数据和存储服务。
- BeeGFS元数据加上存储构建块。
- 一个BeeGFS纯存储组件。

这些组件连接到两个Mellanox Quantum InfiniBand (MQM8700)交换机。此外、还将10个BeeGFS客户端连接到InfiniBand交换机、并用于运行综合基准实用程序。

下图显示了用于验证NetApp解决方案 上的BeeGFS的BeeGFS配置。

[]

BeeGFS文件条带化

并行文件系统的一个优势是能够在多个存储目标之间对各个文件进行条带化、这可能表示同一个或不同底层存储系统上的卷。

在BeeGFS中、您可以按目录和文件配置条带化、以控制每个文件使用的目标数量、并控制每个文件条带使用的机克大小(或块大小)。此配置允许文件系统支持不同类型的工作负载和I/O配置文件、而无需重新配置或重新启动服务。您可以使用`beegfs-ctl`命令行工具或对使用条带化API的应用程序应用条带设置。有关详细信息、请参见的BeeGFS文档 "条带化" 和 "条带化API"。

为了获得最佳性能、我们在整个测试过程中对条带模式进行了调整、并记录了每个测试使用的参数。

IOR带宽测试：多个客户端

IOR带宽测试使用OpenMPI运行合成I/O生成器工具IOR的并行作业(可从获取 "HPC GitHub")连接到一个或多个BeeGFS组件。除非另有说明：

- 所有测试都使用传输大小为1 MiB的直接I/O。
- BeeGFS文件条带化设置为1 MB的机克大小、每个文件一个目标。

以下参数用于IOR、并对区块数进行了调整、以使一个组件的聚合文件大小保持在5 TiB、而三个组件的聚合文件大小保持在40 TiB。

```
mpirun --allow-run-as-root --mca btl tcp -np 48 -map-by node -hostfile
10xnodes ior -b 1024k --posix.odirect -e -t 1024k -s 54613 -z -C -F -E -k
```

下图显示了一个BeeGFS基础(管理、元数据和存储)组件的IOR测试结果。

[]

下图显示了一个BeeGFS元数据+存储组件的IOR测试结果。

[]

下图显示了一个BeeGFS纯存储组件的IOR测试结果。

[]

下图显示了三个BeeGFS组件的IOR测试结果。

[]

正如预期、基础组件与后续元数据+存储组件之间的性能差异可忽略不计。比较元数据+存储构建块和纯存储构建块后、由于用作存储目标的驱动器增加、读取性能略有提高。但是、写入性能没有显著差异。为了提高性能、您可以将多个组件添加到一起、以线性方式扩展性能。

IOR带宽测试：单个客户端

IOR带宽测试使用OpenMPI通过一个高性能GPU服务器运行多个IOR进程、以探索单个客户端可实现的性能。

此测试还会将客户端配置为使用Linux内核页面缓存(`tuneFileCacheType =`原生)时BeeGFS的重新读取行为和性能与默认的`缓冲区`设置进行比较。

原生 缓存模式会在客户端上使用Linux内核页面缓存、从而使重新读取操作可以来自本地内存、而不是通过网络重新传输。

下图显示了三个BeeGFS组件和一个客户端的IOR测试结果。

[]

> 在这些测试中、BeeGFS条带化设置为1 MB的机克大小、每个文件有8个目标。

尽管使用默认缓冲模式时写入和初始读取性能较高、但对于多次重新读取相同数据的工作负载、原生 缓存模式会显著提升性能。对于深度学习等工作负载来说、这种提高的重新读取性能非常重要、因为深度学习会在许多时间内多次重新读取同一数据集。

元数据性能测试

元数据性能测试使用MDTest工具(包含在IOR中)测量BeeGFS的元数据性能。这些测试利用OpenMPI在所有十个客户端节点上运行并行作业。

以下参数用于运行基准测试、其中进程总数从10扩展到320、步长为2倍、文件大小为4 k。

```
mpirun -h 10xnodes –map-by node np $processes mdtest -e 4k -w 4k -i 3 -I
16 -z 3 -b 8 -u
```

元数据性能首先使用一到两个元数据+存储组件进行衡量、以显示性能如何通过添加更多组件进行扩展。

下图显示了一个BeeGFS元数据+存储组件的MDTest结果。

[]

下图显示了包含两个BeeGFS元数据+存储组件的MDTest结果。

[]

功能验证

在验证此架构的过程中、NetApp执行了多项功能测试、其中包括：

- 通过禁用交换机端口使单个客户端InfiniBand端口出现故障。
- 通过禁用交换机端口使单个服务器InfiniBand端口出现故障。
- 使用BMC触发服务器立即关闭。
- 妥善地将节点置于备用状态并将服务故障转移到另一节点。

- 妥善地将节点重新联机、并将服务故障恢复到原始节点。
- 使用PDU关闭其中一个InfiniBand交换机。所有测试均在压力测试期间执行、并在BeeGFS客户端上设置了`ssysSessionChecksEnabled：false`参数。未发现I/O错误或中断。

> 存在一个已知的问题描述 (请参见 "ChangeLog")当BeeGFS客户端/服务器RDMA连接因主接口丢失(如`connInterfacesFile`中所定义)或BeeGFS服务器发生故障而意外中断时、活动客户端I/O可能会挂起多达十分钟、然后才能恢复。如果BeeGFS节点已妥善置于待机状态和待机状态并处于待机状态以进行计划内维护、或者正在使用TCP、则不会发生此问题描述。

NVIDIA DGX A100 SuperPOD验证

NetApp使用一个类似的BeeGFS文件系统验证了用于NVIDIA DGX A100 SuperPOD的存储解决方案 、该文件系统由三个组件组成、并应用了元数据和存储配置文件。资格认定工作涉及使用20台DGX A100 GPU服务器测试此NVA所述的解决方案 、这些服务器运行各种存储、机器学习和深度学习基准。

有关详细信息，请参见 "采用NetApp技术的NVIDIA DGX SuperPOD"。

规模估算准则

BeeGFS解决方案 提供了基于验证测试的性能和容量规模估算建议。

构建块架构的目标是、通过添加多个构建块来满足特定BeeGFS系统的要求、创建一个易于调整大小的解决方案。您可以根据以下准则估算满足环境要求所需的BeeGFS组件的数量和类型。

请记住、这些估计是最佳性能。我们编写并利用了综合基准测试应用程序、以便以实际应用程序可能无法实现的方式优化底层文件系统的使用。

性能规模估算

下表提供了建议的性能规模估算。

配置配置文件	**1 MiB**读取	**1 MiB**写入
元数据+存储	62 GiBps	21 GiBps
仅存储	64 GiBps	21 GiBps

元数据容量估算是根据"经验法则"估算的、即500 GB的容量足以容纳BeeGFS中大约1.5亿个文件。(有关详细信息、请参见的BeeGFS文档 "系统要求")

使用访问控制列表等功能以及每个目录的目录和文件数量也会影响元数据空间的使用速度。存储容量估计值会考虑可用驱动器容量以及RAID 6和XFS开销。

元数据+存储组件的容量规模估算

下表提供了元数据和存储组件的建议容量大小调整。

驱动器大小(**2+2 RAID 1**)元数据卷组	元数据容量(文件数)	驱动器大小(**8+2 RAID 6**)存储卷组	存储容量(文件内容)
1.92 TB	1、797、120、000	1.92 TB	47.91 TB
3.84 TB	3、595、240、000	3.84 TB	95.84 TB
7.68 TB	7、557、120、000	7.68 TB	201.51 TB
15.3 TB	16、156、800、000	15.3 TB	430.84 TB

在估算元数据和存储组件的规模时、您可以使用较小的驱动器作为元数据卷组、而不是存储卷组、从而降低成本。

仅存储组件的容量规模估算

下表提供了仅存储组件的容量规模估算规则。

驱动器大小(**8+2 RAID 6**)存储卷组	存储容量(文件内容)
1.92 TB	59.89 TB
3.84 TB	119.80 TB
7.68 TB	251.89 TB
15.3 TB	538.55 TB

除非启用全局文件锁定、否则将管理服务包含在基本(第一个)组件中所带来的性能和容量开销极小。

性能调整

BeeGFS解决方案 提供了基于验证测试的性能调整建议。

虽然BeeGFS可提供合理的即装即用性能、但NetApp已开发出一组建议的调整参数来最大限度地提高性能。这些参数考虑了底层E系列块节点的功能以及在共享磁盘HA架构中运行BeeGFS所需的任何特殊要求。

文件节点的性能调整

您可以配置的可用调整参数包括：

1. *文件节点的UEFI/BIOS中的系统设置。*为最大限度地提高性能、建议在用作文件节点的服务器型号上配置系统设置。在设置文件节点时、您可以使用由基板管理控制器(BMC)提供的系统设置(UEFI/BIOS)或Redfish API来配置系统设置。

   系统设置因用作文件节点的服务器型号而异。必须根据所使用的服务器型号手动配置这些设置。要了解如何为经过验证的联想SR665文件节点配置系统设置、请参见 "调整文件节点系统设置以提高性能"。

2. *所需配置参数的默认设置。*所需配置参数会影响BeeGFS服务的配置方式以及Pacemaker对E系列卷(块设备)进行格式化和挂载的方式。这些必需的配置参数包括：
   - BeeGFS服务配置参数

您可以根据需要覆盖配置参数的默认设置。有关可根据特定工作负载或用例调整的参数、请参见 "BeeGFS服务配置参数"。

- 卷格式化和挂载参数设置为建议的默认值、只能针对高级使用情形进行调整。默认值将执行以下操作：
  - 根据目标类型(例如管理、元数据或存储)以及底层卷的RAID配置和分段大小优化初始卷格式化。
  - 调整Pacemaker挂载每个卷的方式、以确保所做的更改立即刷新到E系列块节点。这样可以防止在文件节点发生故障且正在进行活动写入时丢失数据。

    有关可根据特定工作负载或用例调整的参数、请参见 "卷格式化和挂载配置参数"。

3. * Linux操作系统中安装在文件节点上的系统设置。*在的步骤4中创建Ansible清单时、您可以覆盖默认Linux操作系统设置 "创建Ansible清单"。

   默认设置用于验证NetApp解决方案 上的BeeGFS、但您可以对其进行更改、以便根据您的特定工作负载或使用情形进行调整。您可以更改的Linux操作系统设置示例包括：

   - E系列块设备上的I/O队列。

     您可以在用作BeeGFS目标的E系列块设备上配置I/O队列、以便：

     - 根据设备类型(NVMe、HDD等)调整计划算法。
     - 增加未处理的请求数。
     - 调整请求大小。
     - 优化预读行为。
   - 虚拟内存设置。

     您可以调整虚拟内存设置、以获得最佳的持续流性能。

   - CPU设置。

     您可以调整CPU频率调速器和其他CPU配置、以获得最高性能。

   - 读取请求大小。

     您可以增加Mellanox HCA的最大读取请求大小。

对块节点进行性能调整

根据应用于特定BeeGFS组件的配置文件、在块节点上配置的卷组会略有变化。例如、对于一个24驱动器EF600块节点：

- 对于单个基础组件、包括BeeGFS管理、元数据和存储服务：
  - 1个2+2 RAID 10卷组、用于BeeGFS管理和元数据服务
  - 2个8+2 RAID 6卷组、用于BeeGFS存储服务
- 对于BeeGFS元数据+存储构建块：
  - 1个2+2 RAID 10卷组、用于BeeGFS元数据服务

  - 2个8+2 RAID 6卷组、用于BeeGFS存储服务
- 仅限BeeGFS存储组件：
  - 2个10+2 RAID 6卷组、用于BeeGFS存储服务

> 与存储相比、BeeGFS对管理和元数据所需的存储空间要少得多、因此可以选择为RAID 10卷组使用较小的驱动器。较小的驱动器应填充到最外侧的驱动器插槽中。有关详细信息，请参见 "部署说明"。

所有这些设置均由基于Ansible的部署进行配置、此外、通常还建议使用其他几种设置来优化性能/行为、其中包括：

- 将全局缓存块大小调整为32 KiB、并将基于需求的缓存刷新调整为80%。
- 禁用自动加载平衡(确保控制器卷分配保持预期状态)。
- 启用读取缓存和禁用预读缓存。
- 通过镜像启用写入缓存并要求使用电池备份、以便在块节点控制器发生故障时保持缓存不变。
- 指定驱动器分配给卷组的顺序、在可用驱动器通道之间平衡I/O。

部署解决方案

部署概述

您可以使用NetApp的第二代BeeGFS组件设计将NetApp上的BeeGFS部署到经验证的文件和块节点。

Ansible集合和角色

您可以使用Ansible在NetApp解决方案 上部署BeeGFS、Ansible是一种常用的IT自动化引擎、用于自动执行应用程序部署。Ansible使用一系列文件、统称为清单、用于为要部署的BeeGFS文件系统建立模型。

Ansible允许NetApp等公司使用Ansible Galax星 上的集合扩展内置功能(请参见 "NetApp E系列BeeGFS集合" ）。集合包括执行某些特定功能或任务(例如创建E系列卷)的模块、以及可以调用多个模块和其他角色的角色。这种自动化方法可减少部署BeeGFS文件系统和底层HA集群所需的时间。此外、它还简化了添加组件以扩展现有文件系统的过程。

有关其他详细信息，请参见 "了解Ansible清单"。

由于在NetApp解决方案 上部署BeeGFS涉及许多步骤、因此NetApp不支持手动部署解决方案。

BeeGFS组件的配置配置文件

部署过程包括以下配置文件：

- 一个基础组件、其中包括管理、元数据和存储服务。
- 包含元数据和存储服务的第二个组件。
- 仅包含存储服务的第三个组件。

这些配置文件展示了NetApp BeeGFS组件的完整建议配置文件。对于每个部署、元数据和存储组件或仅存储服务组件的数量可能因容量和性能要求而有所不同。

部署步骤概述

部署涉及以下高级任务：

硬件部署

1. 以物理方式组装每个组件。
2. 机架和缆线硬件。有关详细过程、请参见 "部署硬件"。

软件部署

1. "设置文件和块节点"。
    - 在文件节点上配置BMC IP
    - 安装受支持的操作系统并在文件节点上配置管理网络
    - 在块节点上配置管理IP

2. "设置Ansible控制节点"。
3. "调整系统性能设置"。
4. "创建Ansible清单"。
5. "为BeeGFS组件定义Ansible清单"。
6. "使用Ansible部署BeeGFS"。
7. "配置BeeGFS客户端"。

> 部署过程包括几个示例、其中需要将文本复制到文件。请密切注意对于任何应针对特定部署进行修改或可修改的内容、任何以"#"或"/"字符表示的实时注释。例如：`beegfs_ha_ntp_server_pools：` This is an example of a comment！ -"pool 0.pool.ntp.org iBurst maxsources 3"-"pool 1.pool.ntp.org iBurst maxsources 3"

了解Ansible清单

开始部署之前、请确保了解如何使用Ansible使用第二代BeeGFS组件设计在NetApp解决方案 上配置和部署BeeGFS。

Ansible清单定义了文件和块节点的配置、并表示要部署的BeeGFS文件系统。清单包括描述所需BeeGFS文件系统的主机、组和变量。可从下载示例清单 "NetApp E系列BeeGFS GitHub"。

Ansible模块和角色

要应用Ansible清单所述的配置、请使用NetApp E系列Ansible集合中提供的各种Ansible模块和角色、尤其是BeeGFS HA 7.2角色(可从获取 "NetApp E系列BeeGFS GitHub")部署端到端解决方案。

NetApp E系列Ansible资料集中的每个角色都是在NetApp解决方案 上完整地端到端部署BeeGFS。这些角色使用NetApp E系列SANtricity 、主机和BeeGFS集合、通过这些集合、您可以使用HA (高可用性)配置BeeGFS文件系统。然后、您可以配置和映射存储、并确保集群存储已准备就绪、可以使用。

虽然角色附带了深入的文档、但部署过程介绍了如何使用角色使用第二代BeeGFS组件设计部署经验证的NetApp架构。

> 尽管部署步骤会尝试提供足够详细的信息、以便事先使用Ansible并不是前提条件、但您应该对Ansible及相关术语有所熟悉。

BeeGFS HA集群的清单布局

使用Ansible清单结构定义BeeGFS HA集群。

任何以前具有Ansible经验的用户都应了解BeeGFS HA角色可通过一种自定义方法来发现每个主机适用的变量(或事实)。要简化Ansible清单的构建、使其能够描述可在多个服务器上运行的资源、需要执行此操作。

Ansible清单通常由`host_vars`和`group_vars`中的文件以及一个`inventory.yml`文件组成、该文件可将主机分配给特定组(并可能将组分配给其他组)。

> 请勿使用本小节中的内容创建任何文件、本小节仅用作示例。

尽管此配置是根据配置配置文件预先确定的、但您应大致了解如何将所有内容作为Ansible清单进行布局、如下所示：

```
# BeeGFS HA (High Availability) cluster inventory.
all:
  children:
    # Ansible group representing all block nodes:
    eseries_storage_systems:
      hosts:
        ictad22a01:
        ictad22a02:
        ictad22a03:
        ictad22a04:
        ictad22a05:
        ictad22a06:
    # Ansible group representing all file nodes:
    ha_cluster:
      children:
        meta_01:  # Group representing a metadata service with ID 01.
          hosts:
            file_node_01:  # This service is preferred on the first file
node.
            file_node_02:  # And can failover to the second file node.
        meta_02:  # Group representing a metadata service with ID 02.
          hosts:
            file_node_02:  # This service is preferred on the second file
node.
            file_node_01: # And can failover to the first file node.
```

对于每个服务、将在`group_vars`下创建一个附加文件、用于描述其配置：

```
# meta_01 - BeeGFS HA Metadata Resource Group
beegfs_ha_beegfs_meta_conf_resource_group_options:
  connMetaPortTCP: 8015
  connMetaPortUDP: 8015
  tuneBindToNumaZone: 0
floating_ips:
  - i1b: <IP>/<SUBNET_MASK>
  - i4b: <IP>/<SUBNET_MASK>
# Type of BeeGFS service the HA resource group will manage.
beegfs_service: metadata # Choices: management, metadata, storage.
# What block node should be used to create a volume for this service:
beegfs_targets:
  ictad22a01:
    eseries_storage_pool_configuration:
      - name: beegfs_m1_m2_m5_m6
        raid_level: raid1
        criteria_drive_count: 4
        owning_controller: A
        common_volume_configuration:
          segment_size_kb: 128
        volumes:
          - size: 21.25
```

通过此布局、可以在一个位置为每个资源定义BeeGFS服务、网络和存储配置。在后台、BeeGFS角色会根据此清单结构为每个文件和块节点聚合所需的配置。有关详细信息、请参见以下博客文章： "NetApp借助Ansible加快了BeeGFS HA的部署"。

> 系统会根据组名称自动配置每个服务的BeeGFS数字和字符串节点ID。因此、除了组名称唯一的一般可逆要求之外、表示BeeGFS服务的组必须以该组所代表的BeeGFS服务类型唯一的数字结尾。例如、允许使用meta_01和stor_01、但不允许使用metadata_01和metada_01。

查看最佳实践

在NetApp解决方案 上部署BeeGFS时、请遵循最佳实践准则。

标准约定

在物理组装和创建Ansible清单文件时、请遵循以下标准约定(有关详细信息、请参见 "创建Ansible清单"）。

- 文件节点主机名按顺序编号(h01-HN)、机架顶部的数字较低、底部的数字较高。

  例如、命名约定`[location][row][rack]HN`的外观如下：`ictad22h01`。

- 每个块节点由两个存储控制器组成、每个控制器都有自己的主机名。

  存储阵列名称用于在Ansible清单中引用整个块存储系统。存储阵列名称应按顺序编号(A01 - A)、各个控制器

的主机名均根据该命名约定派生。

例如、名为`ictad22a01`的块节点通常可以为每个控制器配置主机名、例如`ictad22a01-A`和`ictad22a01-b`、但在Ansible清单中称为`ictad22a01`。

- 同一构建块中的文件和块节点共享相同的编号方案、并在机架中彼此相邻、两个文件节点位于顶部、两个块节点位于其正下方。

  例如、在第一个构建块中、文件节点h01和h02都直接连接到块节点A01和A02。主机名依次为h01、h02、A01和A02。

- 组件会根据主机名按顺序进行安装、因此、编号较低的主机名位于机架顶部、编号较高的主机名位于底部。

  其目的是最大限度地缩短连接到机架交换机顶部的缆线长度、并定义标准部署实践以简化故障排除。对于因机架稳定性问题而不允许这样做的数据中心、当然也允许使用相反方法、即从下往上填充机架。

InfiniBand存储网络配置

每个文件节点上一半的InfiniBand端口用于直接连接到块节点。另一半连接到InfiniBand交换机、并用于BeeGFS客户端-服务器连接。在确定用于BeeGFS客户端和服务器的IPoIB子网的大小时、您必须考虑计算/GPU集群和BeeGFS文件系统的预期增长。如果您必须偏离建议的IP范围、请注意、一个构建块中的每个直接连接都有一个唯一的子网、并且与用于客户端-服务器连接的子网没有重叠。

直接连接

每个构建块中的文件和块节点始终使用下表中的IP进行直接连接。

> 此寻址方案遵循以下规则：第三个八位字节始终为奇数或偶数、具体取决于文件节点是奇数还是偶数。

文件节点	**IB**端口	**IP** 地址	块节点	**IB**端口	物理**IP**	虚拟**IP**
奇数(H1)	i1a	192.168.1.10	奇数(C1)	2a.	192.168.1.100	192.168.1.101
奇数(H1)	I2a.	192.168.3.10	奇数(C1)	2a.	192.168.3.100	192.168.3.101
奇数(H1)	i3a	192.168.5.10	偶数(C2)	2a.	192.168.5.100	192.168.5.101
奇数(H1)	i4a	192.168.7.10	偶数(C2)	2a.	192.168.7.100	192.168.7.101
偶数(H2)	i1a	192.168.2.10	奇数(C1)	2亿	192.168.2.100	192.168.2.101
偶数(H2)	I2a.	192.168.4.10	奇数(C1)	2亿	192.168.4.100	192.168.4.101
偶数(H2)	i3a	192.168.6.10	偶数(C2)	2亿	192.168.6.100	192.168.6.101
偶数(H2)	i4a	192.168.8.10	偶数(C2)	2亿	192.168.8.100	192.168.8.101

BeeGFS客户端**-**服务器**IPoIB**寻址方案**(**两个子网**)**

要使BeeGFS客户端能够使用两个InfiniBand端口、需要两个IPoIB子网、其中一半的BeeGFS服务器服务在每个子网上配置了首选IP、以确保客户端使用两个InfiniBand端口最大限度地提高文件系统的冗余和可能的吞吐量。

每个文件节点都运行多个BeeGFS服务器服务(管理、元数据或存储)。为了使每个服务能够独立故障转移到另一个文件节点、每个服务都配置了唯一的IP地址、这些地址可以在两个节点之间浮动(有时称为逻辑接口或LIF)。

此部署虽然不是必需的、但假定这些连接正在使用以下IPoIB子网范围、并定义了一个标准寻址方案、该方案将应用以下规则：

- 根据文件节点InfiniBand端口是奇数还是奇数、第二个八位字节始终是奇数甚至偶数。
- BeeGFS集群IP始终为`xxx。127.100.yyy`或`xxx.128.100.yyy`。

> 除了用于带内操作系统管理的接口之外、Corosync还可以使用其他接口进行集群检测和同步。这样可以确保丢失一个接口不会导致整个集群关闭。

- BeeGFS管理服务始终为`xxx.yyy.101.0`或`xxx.yyy.102.0`。
- BeeGFS元数据服务始终位于`xxx.yyy.101.zzz`或`xxx.yyy.102.zzz`。
- BeeGFS存储服务始终位于`xxx.yyy.103.zzz`或`xxx.yyy.103.zzz`。
- 范围`100.xxx.1.1`到`100.xxx.99.255`的地址是为客户端预留的。

下表提供了子网A的范围：100.127.0.0/16。

目的	**InfiniBand**端口	**IP**地址或范围
BeeGFS集群IP	i1b	100.127.100.1 - 100.127.100.255
BeeGFS管理	i1b	100.127.101.0
BeeGFS元数据	i1b或i3b	100.127.101.1 - 100.127.101.255
BeeGFS存储	i1b或i3b	100.127.103.1 - 100.127.103.255
BeeGFS客户端	(因客户端而异)	100.127.1.1 - 100.127.99.255

下表提供了子网B的范围：100.128.0.0/16。

目的	**InfiniBand**端口	**IP**地址或范围
BeeGFS集群IP	i4b.	100.128.100.1 - 100.128.100.255
BeeGFS管理	i2b.	100.128.102.0
BeeGFS元数据	i2b或i4b	100.128.102.1 - 100.128.102.255
BeeGFS存储	i2b或i4b	100.128.104.1 - 100.128.104.255
BeeGFS客户端	(因客户端而异)	100.128.1.1 - 100.128.99.255

> 并非上述范围内的所有IP都用于此经过NetApp验证的架构。它们展示了如何预先分配IP地址、以便使用一致的IP寻址方案轻松扩展文件系统。在此方案中、BeeGFS文件节点和服务ID与已知IP范围中的第四个八位字节相对应。如果需要、文件系统可以扩展到255个节点或服务以上。

部署硬件

每个组件都包含两个经过验证的x86文件节点、这些文件节点使用HDR (200 GB) InfiniBand缆线直接连接到两个块节点。

> 由于每个组件都包含两个BeeGFS文件节点、因此要在故障转移集群中建立仲裁、至少需要两个组件。虽然可以配置双节点集群、但此配置存在一些限制、可能会在某些情况下阻止成功进行故障转移。如果需要双节点集群、也可以将第三个设备作为Tiebreaker、但此部署操作步骤 不会涉及这一点。

除非另有说明、否则集群中的每个组件的以下步骤都是相同的、无论该组件是用于运行BeeGFS元数据和存储服务还是仅用于运行存储服务。

步骤

1. 在InfiniBand模式下为每个BeeGFS文件节点配置四个PCIe 4.0 ConnectX-6双端口主机通道适配器(Host Channel Adapter、HCA)、并将其安装在PCIe插槽2、3、5和6中。
2. 为每个BeeGFS块节点配置一个双端口200 GB主机接口卡(HIC)、并在其两个存储控制器中的每个控制器中安装HIC。

   将组件装入机架、使两个BeeGFS文件节点位于BeeGFS块节点上方。下图显示了BeeGFS组件的正确硬件配置(后视图)。

   []

   > 用于生产用例的电源配置通常应使用冗余PSU。

3. 如果需要、请在每个BeeGFS块节点中安装驱动器。
   a. 如果要使用此组件运行BeeGFS元数据和存储服务、而将较小的驱动器用于元数据卷、请验证这些驱动器是否已填充到最外的驱动器插槽中、如下图所示。
   b. 对于所有组件配置、如果驱动器机箱未完全填充、请确保在插槽0–11和12–23中填充相同数量的驱动器、以获得最佳性能。

      []

4. 要为文件和块节点布线、请使用1米InfiniBand HDR 200 GB直连铜缆、以便与下图所示的拓扑结构匹配。

   []

   > 多个组件中的节点不会直接连接。每个组件都应视为一个独立单元、并且各个组件之间的所有通信都通过网络交换机进行。

5. 使用2米(或适当长度)的InfiniBand HDR 200 GB直连铜缆将每个文件节点上的其余InfiniBand端口连接到要用于存储网络的InfiniBand交换机。

   如果正在使用冗余InfiniBand交换机、请使用缆线将下图中以淡绿色突出显示的端口连接到不同的交换机。

   []

6. 根据需要、按照相同的布线准则组装其他组件。

   > 可部署在一个机架中的组件总数取决于每个站点的可用电源和散热。

部署软件

设置文件节点和块节点

虽然大多数软件配置任务都是使用NetApp提供的Ansible集合自动执行的、但您必须在每个服务器的基板管理控制器(Baseboard Management Controller、BMC)上配置网络、并在每个控制器上配置管理端口。

设置文件节点

1. 在每个服务器的基板管理控制器(BMC)上配置网络连接。

   要了解如何为经过验证的联想SR665文件节点配置网络、请参见 "联想ThinkSystem文档"。

   > 基板管理控制器(Baseboard Management Controller、BMC)有时也称为服务处理器、它是内置于各种服务器平台中的带外管理功能的通用名称、即使操作系统未安装或无法访问、也可以提供远程访问。供应商通常会使用自己的品牌推广此功能。例如、在联想SR665上、BMC称为_Lenovo Xicity1控制器(XCC)_。

2. 配置系统设置以实现最高性能。

   您可以使用UEFI设置(以前称为BIOS)或许多BMC提供的Redfish API配置系统设置。系统设置因用作文件节点的服务器型号而异。

   要了解如何为经过验证的联想SR665文件节点配置系统设置、请参见 "调整系统性能设置"。

3. 安装Red Hat 8.4并配置用于管理操作系统的主机名和网络端口、包括来自Ansible控制节点的SSH连接。

   此时、请勿在任何InfiniBand端口上配置IP。

   > 尽管严格要求不高、但后续章节假定主机名按顺序编号(例如H1-HN)、并提及应在奇数主机上完成的任务、而不是偶数主机上完成的任务。

4. 使用RedHat订阅管理器注册并订阅系统、以便可以从Red Hat官方存储库安装所需的软件包、并将更新限制为受支持的Red Hat版本：`ssubscription-manager release -set=8.4`。有关说明，请参见 "如何注册和订阅RHEL系统" 和 "如何限制更新"。

5. 启用包含高可用性所需软件包的Red Hat存储库。

   ```
   subscription-manager repo-override --repo=rhel-8-for-x86_64
   -highavailability-rpms --add=enabled:1
   ```

6. 将所有ConnectX-6 HCA固件更新为中建议的版本 "技术要求"。

   要完成此更新、可以下载并运行捆绑了建议固件的lxup工具版本。您可以从下载此工具 "lxup—更新和查询实用程序"。

设置块节点

通过在每个控制器上配置管理端口来设置EF600块节点。

1. 在每个EF600控制器上配置管理端口。

   有关配置端口的说明、请转至 "E系列文档中心"。

2. (可选)设置每个系统的存储阵列名称。

   通过设置名称、可以更轻松地在后续章节中引用每个系统。有关设置阵列名称的说明、请转到 "E系列文档中心"。

> 尽管严格要求不严格、但后续主题假定存储阵列名称按顺序编号(例如C1 - CN)、并提及在奇数系统甚至编号系统上应完成的步骤。

设置Ansible控制节点

要设置Ansible控制节点、您必须确定能够通过网络访问可用于配置解决方案 的所有文件和块节点的管理端口的虚拟机或物理机。

以下步骤已在CentOS 8.4上进行了测试。有关首选Linux分发版的特定步骤、请参见 "Ansible文档"。

1. 安装Python 3.9并确保安装了正确版本的`pip`。

```
sudo dnf install python3.9 -y
sudo dnf install python39-pip
sudo dnf install sshpass
```

2. 创建符号链接、确保每当调用`python3`或`python`时使用Python 3.9二进制文件。

```
sudo ln -sf /usr/bin/python3.9 /usr/bin/python3
sudo ln -sf /usr/bin/python3 /usr/bin/python
```

3. 安装NetApp BeeGFS集合所需的Python软件包。

```
python3 -m pip install ansible cryptography netaddr
```

> 要确保您安装的是受支持的Ansible版本以及所有必需的Python软件包、请参阅BeeGFS集合的自述文件。中还说明了支持的版本 "技术要求"。

4. 验证是否已安装正确版本的Ansible和Python。

```
ansible --version
ansible [core 2.11.6]
  config file = None
  configured module search path = ['/root/.ansible/plugins/modules',
'/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/local/lib/python3.9/site-
packages/ansible
  ansible collection location =
/root/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/local/bin/ansible
  python version = 3.9.2 (default, Mar 10 2021, 17:29:56) [GCC 8.4.1
20200928 (Red Hat 8.4.1-1)]
  jinja version = 3.0.2
  libyaml = True
```

5. 将用于描述BeeGFS部署的Ansible清单存储在Git或BitBucket等源控制系统中、然后安装Git以与这些系统进行交互。

```
sudo dnf install git -y
```

6. 设置无密码SSH。这是允许Ansible从Ansible控制节点访问远程BeeGFS文件节点的最简单方法。
   a. 如果需要、在Ansible控制节点上、使用`ssh-keygen`生成一对公有 密钥
   b. 使用`ssh-copy-id <ip_or_hostname>`为每个文件节点设置无密码SSH

      Do * not* set up passworwless SSH to the block nodes.这既不受支持、也不是必需的。

7. 使用Ansible GALAXY安装中列出的BeeGFS集合版本 "技术要求"。

   此安装还包括其他Ansible依赖关系、例如NetApp SANtricity 软件和主机集合。

```
ansible-galaxy collection install netapp_eseries.beegfs:==3.0.1
```

创建Ansible清单

要定义文件和块节点的配置、请创建一个Ansible清单、用于表示要部署的BeeGFS文件系统。清单包括描述所需BeeGFS文件系统的主机、组和变量。

第1步：定义所有组件的配置

定义适用场景 所有组件的配置、无论您可以单独应用于这些组件的配置文件如何。

开始之前

- 使用BitBucket或Git等源控制系统存储包含Ansible清单和攻略手册文件的目录的内容。

- 创建一个`.gitignore`文件、用于指定Git应忽略的文件。这有助于避免在Git中存储大型文件。

步骤

1. 在Ansible控制节点上、确定要用于存储Ansible清单和攻略手册文件的目录。

   除非另有说明、否则在此步骤中创建的所有文件和目录以及后续步骤均会相对于此目录创建。

2. 创建以下子目录：

   `host_vars`

   `group_vars`

   软件包

第**2**步：为单个文件和块节点定义配置

定义适用场景 单个文件节点和单个组件节点的配置。

1. 在`host_vars/下、为名为<HOSTNAME>.yml`的每个BeeGFS文件节点创建一个文件、其中包含以下内容、请特别注意有关BeeGFS集群IP和以奇数与偶数结尾的主机名要填充的内容的注释。

   最初、文件节点接口名称与此处列出的名称匹配(例如ib0或ibs1f0)。这些自定义名称在中进行配置 [Step 4: Define configuration that should apply to all file nodes]。

```
ansible_host: “<MANAGEMENT_IP>”
eseries_ipoib_interfaces:  # Used to configure BeeGFS cluster IP
addresses.
  - name: i1b
    address: 100.127.100. <NUMBER_FROM_HOSTNAME>/16
  - name: i4b
    address: 100.128.100. <NUMBER_FROM_HOSTNAME>/16
beegfs_ha_cluster_node_ips:
  - <MANAGEMENT_IP>
  - <i1b_BEEGFS_CLUSTER_IP>
  - <i4b_BEEGFS_CLUSTER_IP>
# NVMe over InfiniBand storage communication protocol information
# For odd numbered file nodes (i.e., h01, h03, ..):
eseries_nvme_ib_interfaces:
  - name: i1a
    address: 192.168.1.10/24
    configure: true
  - name: i2a
    address: 192.168.3.10/24
    configure: true
  - name: i3a
    address: 192.168.5.10/24
    configure: true
  - name: i4a
    address: 192.168.7.10/24
    configure: true
# For even numbered file nodes (i.e., h02, h04, ..):
# NVMe over InfiniBand storage communication protocol information
eseries_nvme_ib_interfaces:
  - name: i1a
    address: 192.168.2.10/24
    configure: true
  - name: i2a
    address: 192.168.4.10/24
    configure: true
  - name: i3a
    address: 192.168.6.10/24
    configure: true
  - name: i4a
    address: 192.168.8.10/24
    configure: true
```

如果您已部署BeeGFS集群、则必须先停止此集群、然后才能添加或更改静态配置的IP地址、包括用于NVMe/IB的集群IP和IP。这是必需的、以便这些更改能够正确生效、并且不会中断集群操作。

2. 在`host_vars/下、为名为<HOSTNAME>.yml`的每个BeeGFS块节点创建一个文件、并使用以下内容填充该文件。

   请特别注意有关以奇数与偶数结尾的存储阵列名称要填充的内容的注释。

   对于每个块节点、创建一个文件并为两个控制器(通常为A)之一指定`<management_IP>`。

```
eseries_system_name: <STORAGE_ARRAY_NAME>
eseries_system_api_url: https://<MANAGEMENT_IP>:8443/devmgr/v2/
eseries_initiator_protocol: nvme_ib
# For odd numbered block nodes (i.e., a01, a03, ..):
eseries_controller_nvme_ib_port:
  controller_a:
    - 192.168.1.101
    - 192.168.2.101
    - 192.168.1.100
    - 192.168.2.100
  controller_b:
    - 192.168.3.101
    - 192.168.4.101
    - 192.168.3.100
    - 192.168.4.100
# For even numbered block nodes (i.e., a02, a04, ..):
eseries_controller_nvme_ib_port:
  controller_a:
    - 192.168.5.101
    - 192.168.6.101
    - 192.168.5.100
    - 192.168.6.100
  controller_b:
    - 192.168.7.101
    - 192.168.8.101
    - 192.168.7.100
    - 192.168.8.100
```

第**3**步：定义应应用于所有文件和块节点的配置

您可以使用与组对应的文件名在`group_vars`下定义一组主机的通用配置。这样可以防止在多个位置重复使用共享配置。

主机可以位于多个组中、运行时、Ansible会根据特定主机的可变优先级规则选择哪些变量适用于该主机。(有关这些规则的详细信息、请参见的Ansible文档 "使用变量"）

主机到组分配在实际可Ansible清单文件中进行定义、该文件在该操作步骤 接近末尾时创建。

在Ansible中、可以在名为`All`的组中定义要应用于所有主机的任何配置。使用以下内容创建文件`group_vars/all.yml`：

```
ansible_python_interpreter: /usr/bin/python3
beegfs_ha_ntp_server_pools:  # Modify the NTP server addressess if
desired.
  - "pool 0.pool.ntp.org iburst maxsources 3"
  - "pool 1.pool.ntp.org iburst maxsources 3"
```

第**4**步：定义应应用于所有文件节点的配置

文件节点的共享配置在名为`ha_cluster`的组中定义。本节中的步骤将构建应包含在`group_vars/ha_cluster.yml`文件中的配置。

步骤

1. 在文件顶部、定义默认值、包括用作文件节点上的`sUdo`用户的密码。

```
### ha_cluster Ansible group inventory file.
# Place all default/common variables for BeeGFS HA cluster resources
below.
### Cluster node defaults
ansible_ssh_user: root
ansible_become_password: <PASSWORD>
eseries_ipoib_default_hook_templates:
  - 99-multihoming.j2 # This is required when configuring additional
static IPs (for example cluster IPs) when multiple IB ports are in the
same IPoIB subnet.
# If the following options are specified, then Ansible will
automatically reboot nodes when necessary for changes to take effect:
eseries_common_allow_host_reboot: true
eseries_common_reboot_test_command: "systemctl --state=active,exited |
grep eseries_nvme_ib.service"
```

> 尤其是在生产环境中、不要以纯文本格式存储密码。请改用Ansible Vault (请参见 "使用Ansible Vault加密内容")或运行攻略手册时的`-ask-pass-pass`选项。如果`Ansible_ssh_user`已是`root`、则可以选择省略`Ansible变为密码`。

2. (可选)配置高可用性(HA)集群的名称、并为集群内通信指定用户。

   如果要修改专用IP寻址方案、则还必须更新默认值`beegfs_ha_mgmtd_float_IP`。这必须与您稍后为BeeGFS管理资源组配置的内容匹配。

   使用`beegfs_ha_alert_email_list`指定一个或多个应接收集群事件警报的电子邮件。

```
### Cluster information
beegfs_ha_firewall_configure: True
eseries_beegfs_ha_disable_selinux: True
eseries_selinux_state: disabled
# The following variables should be adjusted depending on the desired
configuration:
beegfs_ha_cluster_name: hacluster                  # BeeGFS HA cluster
name.
beegfs_ha_cluster_username: hacluster              # BeeGFS HA cluster
username.
beegfs_ha_cluster_password: hapassword             # BeeGFS HA cluster
username's password.
beegfs_ha_cluster_password_sha512_salt: randomSalt # BeeGFS HA cluster
username's password salt.
beegfs_ha_mgmtd_floating_ip: 100.127.101.0         # BeeGFS management
service IP address.
# Email Alerts Configuration
beegfs_ha_enable_alerts: True
beegfs_ha_alert_email_list: ["email@example.com"]  # E-mail recipient
list for notifications when BeeGFS HA resources change or fail.  Often a
distribution list for the team responsible for managing the cluster.
beegfs_ha_alert_conf_ha_group_options:
      mydomain: “example.com”
# The mydomain parameter specifies the local internet domain name. This
is optional when the cluster nodes have fully qualified hostnames (i.e.
host.example.com).
# Adjusting the following parameters is optional:
beegfs_ha_alert_timestamp_format: "%Y-%m-%d %H:%M:%S.%N" #%H:%M:%S.%N
beegfs_ha_alert_verbosity: 3
#  1) high-level node activity
#  3) high-level node activity + fencing action information + resources
(filter on X-monitor)
#  5) high-level node activity + fencing action information + resources
```

> 虽然`冗余、但当您将BeeGFS文件系统扩展到单个HA集群之外时、`beegfs_ha_mgmtd_floating_ip非常重要。部署后续HA集群时无需额外的BeeGFS管理服务、并指向第一个集群提供的管理服务。

3. 配置隔离代理。(有关详细信息、请参见 "在Red Hat High Availability集群中配置隔离") 以下输出显示了配置通用隔离代理的示例。选择以下选项之一。

   在此步骤中、请注意：

   - 默认情况下、隔离处于启用状态、但您需要配置隔离_agent_。
   - 在`PCMK_HOST_MAP`或`PCMK_HOST_LIST`中指定的`<HOSTNAME>`必须与Ansible清单中的主机

名相对应。

- 不支持在不使用隔离的情况下运行BeeGFS集群、尤其是在生产环境中。这在很大程度上是为了确保当BeeGFS服务(包括块设备等任何资源依赖关系)因问题描述 而发生故障转移时、不会存在多个节点并发访问导致文件系统损坏或其他不希望或意外行为的风险。如果必须禁用隔离、请参阅BeeGFS HA角色的入门指南中的一般说明、并在`ha_cluster_crm_config_options["stonith-enabled"]`中将`beegfs_ha_cluster.yml`设置为false。
- 可以使用多个节点级别的隔离设备、BeeGFS HA角色可以配置Red Hat HA软件包存储库中可用的任何隔离代理。如果可能、请使用通过不间断电源(UPS)或机架配电单元(rPDU)工作的隔离代理。 因为在某些故障情形下、某些隔离代理(如基板管理控制器(BMC)或服务器中内置的其他无人值守设备)可能无法响应隔离请求。

```
### Fencing configuration:
# OPTION 1: To enable fencing using APC Power Distribution Units
(PDUs):
beegfs_ha_fencing_agents:
 fence_apc:
   - ipaddr: <PDU_IP_ADDRESS>
     login: <PDU_USERNAME>
     passwd: <PDU_PASSWORD>
     pcmk_host_map:
"<HOSTNAME>:<PDU_PORT>,<PDU_PORT>;<HOSTNAME>:<PDU_PORT>,<PDU_PORT>"
# OPTION 2: To enable fencing using the Redfish APIs provided by the
Lenovo XCC (and other BMCs):
redfish: &redfish
  username: <BMC_USERNAME>
  password: <BMC_PASSWORD>
  ssl_insecure: 1 # If a valid SSL certificate is not available
specify “1”.
beegfs_ha_fencing_agents:
  fence_redfish:
    - pcmk_host_list: <HOSTNAME>
      ip: <BMC_IP>
      <<: *redfish
    - pcmk_host_list: <HOSTNAME>
      ip: <BMC_IP>
      <<: *redfish
# For details on configuring other fencing agents see
https://access.redhat.com/documentation/en-
us/red_hat_enterprise_linux/8/html/configuring_and_managing_high_avai
lability_clusters/assembly_configuring-fencing-configuring-and-
managing-high-availability-clusters.
```

4. 在Linux操作系统中启用建议的性能调整。

   虽然许多用户发现性能参数的默认设置通常运行良好、但您也可以选择更改特定工作负载的默认设置。因此、这些建议包含在BeeGFS角色中、但默认情况下不会启用、以确保用户了解应用于其文件系统的调整。

要启用性能调整、请指定：

```
### Performance Configuration:
beegfs_ha_enable_performance_tuning: True
```

5. (可选)您可以根据需要调整Linux操作系统中的性能调整参数。

   有关可调整的可用调整参数的完整列表、请参见中BeeGFS HA角色的"性能调整默认值"部分 "E系列BeeGFS GitHub站点"。可以覆盖此文件或单个节点的`host_vars`文件中集群中所有节点的默认值。

6. 要在块节点和文件节点之间实现完整的200 GB/HDR连接、请使用Mellanox Open Fabrics Enterprise Distribution (MLNx_OFED)中的Open Subnet Manager (OpenSM)软件包。(收件箱`opensm`软件包不支持所需的虚拟化功能。) 尽管支持使用Ansible进行部署、但您必须先将所需的软件包下载到用于运行BeeGFS角色的Ansible控制节点。

   a. 使用`cURL`或您所需的工具、将技术要求部分所列OpenSM版本的软件包从Mellanox的网站下载到`packages/`目录。例如：

   ```
   curl -o packages/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
   0.1.54103.x86_64.rpm
   https://linux.mellanox.com/public/repo/mlnx_ofed/5.4-
   1.0.3.0/rhel8.4/x86_64/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
   0.1.54103.x86_64.rpm

   curl -o packages/opensm-5.9.0.MLNX20210617.c9f2ade-
   0.1.54103.x86_64.rpm
   https://linux.mellanox.com/public/repo/mlnx_ofed/5.4-
   1.0.3.0/rhel8.4/x86_64/opensm-5.9.0.MLNX20210617.c9f2ade-
   0.1.54103.x86_64.rpm
   ```

   b. 在`group_vars/ha_cluster.yml`中填充以下参数(根据需要调整软件包)：

```
### OpenSM package and configuration information
eseries_ib_opensm_allow_upgrades: true
eseries_ib_opensm_skip_package_validation: true
eseries_ib_opensm_rhel_packages: []
eseries_ib_opensm_custom_packages:
  install:
    - files:
        add:
          "packages/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm": "/tmp/"
          "packages/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm": "/tmp/"
    - packages:
        add:
          - /tmp/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
          - /tmp/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
  uninstall:
    - packages:
        remove:
          - opensm
          - opensm-libs
      files:
        remove:
          - /tmp/opensm-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
          - /tmp/opensm-libs-5.9.0.MLNX20210617.c9f2ade-
0.1.54103.x86_64.rpm
eseries_ib_opensm_options:
  virt_enabled: "2"
```

7. 配置`udev`规则、以确保逻辑InfiniBand端口标识符与底层PCIe设备的映射一致。

   `udev`规则对于用作BeeGFS文件节点的每个服务器平台的PCIe拓扑来说必须是唯一的。

   对于已验证的文件节点、请使用以下值：

```
### Ensure Consistent Logical IB Port Numbering
# OPTION 1: Lenovo SR665 PCIe address-to-logical IB port mapping:
eseries_ipoib_udev_rules:
  "0000:41:00.0": i1a
  "0000:41:00.1": i1b
  "0000:01:00.0": i2a
  "0000:01:00.1": i2b
  "0000:a1:00.0": i3a
  "0000:a1:00.1": i3b
  "0000:81:00.0": i4a
  "0000:81:00.1": i4b

# Note: At this time no other x86 servers have been qualified.
Configuration for future qualified file nodes will be added here.
```

8. (可选)更新元数据目标选择算法。

```
beegfs_ha_beegfs_meta_conf_ha_group_options:
  tuneTargetChooser: randomrobin
```

> 在验证测试中、通常会使用`Randomrobin`来确保测试文件在性能基准测试期间均匀分布在所有BeeGFS存储目标上(有关基准测试的详细信息、请参见BeeGFS站点 "对BeeGFS系统进行基准测试"）。在实际使用情况下、这可能会导致编号较低的目标达到发生原因 、从而比编号较高的目标更快地达到填充速度。已显示省略`randomrobin`以及仅使用默认值`randomized`值可在仍利用所有可用目标的情况下提供良好的性能。

第**5**步：定义通用块节点的配置

块节点的共享配置在名为`Eseries_storage_systems`的组中定义。本节中的步骤构建了应包含在`group_vars/eseries_storage_systems.yml`文件中的配置。

步骤

1. 将Ansible连接设置为local、提供系统密码、并指定是否应验证SSL证书。(通常、Ansible使用SSH连接到受管主机、但对于用作块节点的NetApp E系列存储系统、模块使用REST API进行通信。) 在文件顶部、添加以下内容：

```
### eseries_storage_systems Ansible group inventory file.
# Place all default/common variables for NetApp E-Series Storage Systems
here:
ansible_connection: local
eseries_system_password: <PASSWORD>
eseries_validate_certs: false
```

> 不建议以纯文本格式列出任何密码。使用`-ext-vars`运行Ansible时、请使用Ansible vault或提供`E系列系统密码`。

2. 为确保获得最佳性能、请在中安装为块节点列出的版本 "技术要求"。

   从下载相应的文件 "NetApp 支持站点"。您可以手动升级它们、也可以将其包含在Ansible控制节点的`packages/`目录中、然后在`Esery_storage_systems.yml`中填充以下参数以使用Ansible进行升级：

```
# Firmware, NVSRAM, and Drive Firmware (modify the filenames as needed):
eseries_firmware_firmware: "packages/RCB_11.70.2_6000_61b1131d.dlp"
eseries_firmware_nvsram: "packages/N6000-872834-D06.dlp"
```

3. 从下载并安装可用于块节点中安装的驱动器的最新驱动器固件 "NetApp 支持站点"。您可以手动升级它们、也可以将其包含在Ansible控制节点的`packages/`目录中、然后在`Esery_storage_systems.yml`中填充以下参数以使用Ansible进行升级：

```
eseries_drive_firmware_firmware_list:
  - "packages/<FILENAME>.dlp"
eseries_drive_firmware_upgrade_drives_online: true
```

> 将`esery_drive_firmware_upgrade_drives_online`设置为`false`可以加快升级速度、但在部署BeeGFS之前不应执行此操作。这是因为该设置要求在升级之前停止驱动器的所有I/O、以避免应用程序错误。尽管在配置卷之前执行联机驱动器固件升级仍很快、但我们建议您始终将此值设置为`true`以避免稍后出现问题。

4. 要优化性能、请对全局配置进行以下更改：

```
# Global Configuration Defaults
eseries_system_cache_block_size: 32768
eseries_system_cache_flush_threshold: 80
eseries_system_default_host_type: linux dm-mp
eseries_system_autoload_balance: disabled
eseries_system_host_connectivity_reporting: disabled
eseries_system_controller_shelf_id: 99 # Required.
```

5. 要确保最佳卷配置和行为、请指定以下参数：

```
# Storage Provisioning Defaults
eseries_volume_size_unit: pct
eseries_volume_read_cache_enable: true
eseries_volume_read_ahead_enable: false
eseries_volume_write_cache_enable: true
eseries_volume_write_cache_mirror_enable: true
eseries_volume_cache_without_batteries: false
eseries_storage_pool_usable_drives:
"99:0,99:23,99:1,99:22,99:2,99:21,99:3,99:20,99:4,99:19,99:5,99:18,99:6,
99:17,99:7,99:16,99:8,99:15,99:9,99:14,99:10,99:13,99:11,99:12"
```

> 为`E系列_storage_pool_usable_drives`指定的值特定于NetApp EF600块节点、并控制驱动器分配给新卷组的顺序。此顺序可确保每个组的I/O在后端驱动器通道之间均匀分布。

为**BeeGFS**组件定义**Ansible**清单

定义了常规Ansible清单结构后、为BeeGFS文件系统中的每个组件定义配置。

这些部署说明演示了如何部署文件系统、该文件系统由包括管理、元数据和存储服务在内的基础组件、包含元数据和存储服务的第二个组件以及仅存储的第三个组件组成。

这些步骤旨在显示一系列典型配置文件、您可以使用这些配置文件来配置NetApp BeeGFS组件、以满足整个BeeGFS文件系统的要求。

> 在本节及后续章节中、根据需要进行调整、以构建表示要部署的BeeGFS文件系统的清单。特别是、请使用表示每个块或文件节点的Ansible主机名以及存储网络所需的IP寻址方案、以确保它可以扩展到BeeGFS文件节点和客户端的数量。

第**1**步：创建**Ansible**清单文件

步骤

1. 创建新的`inventory.yml`文件、然后插入以下参数、根据需要替换`Esery_storage_systems`下的主机、以表示部署中的块节点。这些名称应与`host_vars/<filename>.yml`使用的名称相对应。

```
# BeeGFS HA (High Availability) cluster inventory.
all:
  children:
    # Ansible group representing all block nodes:
    eseries_storage_systems:
      hosts:
        ictad22a01:
        ictad22a02:
        ictad22a03:
        ictad22a04:
        ictad22a05:
        ictad22a06:
    # Ansible group representing all file nodes:
    ha_cluster:
      children:
```

在后续各节中、您将在`ha_cluster`下创建其他Ansible组、这些组表示您要在集群中运行的BeeGFS服务。

第2步：为管理、元数据和存储组件配置清单

集群或基础构建块中的第一个组件必须包括BeeGFS管理服务以及元数据和存储服务：

步骤

1. 在`inventory.yml`中、在`ha_cluster下填充以下参数：子项`：

```
      # ictad22h01/ictad22h02 HA Pair (mgmt/meta/storage building
block):
        mgmt:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_01:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_01:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_02:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_02:
```

```
          hosts:
            ictad22h01:
            ictad22h02:
        meta_03:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_03:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_04:
          hosts:
            ictad22h01:
            ictad22h02:
        stor_04:
          hosts:
            ictad22h01:
            ictad22h02:
        meta_05:
          hosts:
            ictad22h02:
            ictad22h01:
        stor_05:
          hosts:
            ictad22h02:
            ictad22h01:
        meta_06:
          hosts:
            ictad22h02:
            ictad22h01:
        stor_06:
          hosts:
            ictad22h02:
            ictad22h01:
        meta_07:
          hosts:
            ictad22h02:
            ictad22h01:
        stor_07:
          hosts:
            ictad22h02:
            ictad22h01:
        meta_08:
          hosts:
            ictad22h02:
```

```
            ictad22h01:
        stor_08:
          hosts:
            ictad22h02:
            ictad22h01:
```

2. 创建文件`group_vars/mgmt.yml`并包含以下内容：

```
# mgmt - BeeGFS HA Management Resource Group
# OPTIONAL: Override default BeeGFS management configuration:
# beegfs_ha_beegfs_mgmtd_conf_resource_group_options:
#  <beegfs-mgmt.conf:key>:<beegfs-mgmt.conf:value>
floating_ips:
  - i1b: 100.127.101.0/16
  - i2b: 100.128.102.0/16
beegfs_service: management
beegfs_targets:
  ictad22a01:
    eseries_storage_pool_configuration:
      - name: beegfs_m1_m2_m5_m6
        raid_level: raid1
        criteria_drive_count: 4
        common_volume_configuration:
          segment_size_kb:  128
        volumes:
          - size: 1
            owning_controller: A
```

3. 在`group_vars/`下、使用以下模板为资源组`mETA_01`到`mETA_08`创建文件、然后参考下表填写每个服务的占位值：

```
# meta_0X - BeeGFS HA Metadata Resource Group
beegfs_ha_beegfs_meta_conf_resource_group_options:
  connMetaPortTCP: <PORT>
  connMetaPortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET> # Example: i1b:192.168.120.1/16
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: metadata
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid1
        criteria_drive_count: 4
        common_volume_configuration:
          segment_size_kb:  128
        volumes:
          - size: 21.25 # SEE NOTE BELOW!
            owning_controller: <OWNING CONTROLLER>
```

卷大小以整个存储池(也称为卷组)的百分比形式指定。NetApp强烈建议您在每个池中保留一些可用容量、以便为SSD过度配置留出空间(有关详细信息、请参见 "NetApp EF600阵列简介"）。存储池`beegfs_m1_m2_m3_m3_m6`也会将池容量的1%分配给管理服务。因此、对于存储池中的元数据卷`beegfs_m1_m2_m5_m6`、如果使用1.92 TB或3.84 TB驱动器、请将此值设置为`21.25`；对于7.65 TB驱动器、请将此值设置为`22.25`；对于15.3 TB驱动器、请将此值设置为`23.75`。

文件名	**Port**	浮动**IP**	**NUMA**区域	块节点	存储池	所属控制器
meta_01.yml	8015	i1b ：100.127.1.1.1/16 i2b ：100.128.102.1/16	0	ictad22a01	beegfs_m1_m2_m3_m3_m6	答
meta_02.yml	8025	i2b ：100.128.102.2/16 i1b ：100.127.101.2/16	0	ictad22a01	beegfs_m1_m2_m3_m3_m6	B
meta_03.yml	8035	i3b ：100.127.101.3/16 i4b ：100.128.102.3/16	1.	ictad22a02	beegfs_m3_m4_m7_m8	答

文件名	**Port**	浮动**IP**	**NUMA**区域	块节点	存储池	所属控制器
meta_04.yml	8045	i4b：100.128.102.4/16 i3b：100.127.101.4/16	1.	ictad22a02	beegfs_m3_m4_m7_m8	B
meta_05.yml	8055	i1b：100.127.101.5/16 i2b：100.128.102.5/16	0	ictad22a01	beegfs_m1_m2_m3_m3_m6	答
meta_06.yml	8065	i2b：100.128.102.6/16 i1b：100.127.101.6/16	0	ictad22a01	beegfs_m1_m2_m3_m3_m6	B
meta_07.yml	8075	i3b：100.127.101.7/16 i4b：100.128.102.7/16	1.	ictad22a02	beegfs_m3_m4_m7_m8	答
meta_08.yml	8085	i4b：100.128.102.8/16 i3b：100.127.101.8/16	1.	ictad22a02	beegfs_m3_m4_m7_m8	B

4. 在`group_vars/`下、使用以下模板为资源组`stor_01`到`stor_08`创建文件、然后参考以下示例填写每个服务的占位值：

```
# stor_0X - BeeGFS HA Storage Resource
Groupbeegfs_ha_beegfs_storage_conf_resource_group_options:
  connStoragePortTCP: <PORT>
  connStoragePortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: storage
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid6
        criteria_drive_count: 10
        common_volume_configuration:
          segment_size_kb: 512        volumes:
          - size: 21.50 # See note below!             owning_controller:
<OWNING CONTROLLER>
          - size: 21.50            owning_controller: <OWNING
CONTROLLER>
```

有关要使用的正确大小、请参见 "建议的存储池过度配置百分比"。

文件名	Port	浮动IP	NUMA区域	块节点	存储池	所属控制器
STOR_01.yml	8013	i1b：100.127.103.1/16 i2b：100.128.104.1/16	0	ictad22a01	beegfs_s1_s2	答
STOR_02.yml	8023	i2b：100.128.104.2/16 i1b：100.127.103.2/16	0	ictad22a01	beegfs_s1_s2	B
STOR_03.yml	8033	i3b：100.127.103.3/16 i4b：100.128.104.3/16	1.	ictad22a02	beegfs_s3_s4.	答
STOR_04.yml	8043	i4b：100.128.104.4/16 i3b：100.127.103.4/16	1.	ictad22a02	beegfs_s3_s4.	B

文件名	Port	浮动IP	NUMA区域	块节点	存储池	所属控制器
STOR_05.yml	8053	i1b：100.127.103.5/16 i2b：100.128.104.5/16	0	ictad22a01	beegfs_s5_s6	答
STOR_06.yml	8063	i2b：100.128.104.6/16 i1b：100.127.103.6/16	0	ictad22a01	beegfs_s5_s6	B
STOR_07.yml	8073.	i3b：100.127.103.7/16 i4b：100.128.104.7/16	1.	ictad22a02	beegfs_s7_s8.	答
STOR_08.yml	8083.	i4b：100.128.104.8/16 i3b：100.127.103.8/16	1.	ictad22a02	beegfs_s7_s8.	B

第3步：为元数据+存储构建块配置清单

以下步骤介绍如何为BeeGFS元数据+存储构建块设置Ansible清单。

步骤

1. 在`inventory.yml`中、在现有配置下填充以下参数：

```
        meta_09:
          hosts:
            ictad22h03:
            ictad22h04:
        stor_09:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_10:
          hosts:
            ictad22h03:
            ictad22h04:
        stor_10:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_11:
```

```
          hosts:
            ictad22h03:
            ictad22h04:
        stor_11:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_12:
          hosts:
            ictad22h03:
            ictad22h04:
        stor_12:
          hosts:
            ictad22h03:
            ictad22h04:
        meta_13:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_13:
          hosts:
            ictad22h04:
            ictad22h03:
        meta_14:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_14:
          hosts:
            ictad22h04:
            ictad22h03:
        meta_15:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_15:
          hosts:
            ictad22h04:
            ictad22h03:
        meta_16:
          hosts:
            ictad22h04:
            ictad22h03:
        stor_16:
          hosts:
            ictad22h04:
```

```
            ictad22h03:
```

2. 在`group_vars/`下、使用以下模板为资源组`mETA_09`到`mETA_16`创建文件、然后参考以下示例填写每个服务的占位值：

```
# meta_0X - BeeGFS HA Metadata Resource Group
beegfs_ha_beegfs_meta_conf_resource_group_options:
  connMetaPortTCP: <PORT>
  connMetaPortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: metadata
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid1
        criteria_drive_count: 4
        common_volume_configuration:
          segment_size_kb: 128
        volumes:
          - size: 21.5 # SEE NOTE BELOW!
            owning_controller: <OWNING CONTROLLER>
```

有关要使用的正确大小、请参见 "建议的存储池过度配置百分比"。

文件名	**Port**	浮动**IP**	**NUMA**区域	块节点	存储池	所属控制器
meta_09.yml	8015	i1b：100.127.101.9/16 i2b：100.128.102.9/16	0	ictad22a03	Beegfs_M9_M10_M13_M14	答
meta_10.yml	8025	i2b：100.128.102.10/16 i1b：100.127.101.10/16	0	ictad22a03	Beegfs_M9_M10_M13_M14	B
meta_11.yml	8035	i3b：100.127.2.1.1/16 i4b：100.128.102.11/16	1.	ictad22a04	Beegfs_M11_M12_M15_16	答

文件名	**Port**	浮动**IP**	**NUMA**区域	块节点	存储池	所属控制器
meta_12.yml	8045	i4b：100.128.102.12或16 i3b：100.127.1.1.12或16	1.	ictad22a04	Beegfs_M11_M12_M15_16	B
meta_13.yml	8055	i1b：100.127.102.13/16 i2b：100.128.102.13/16	0	ictad22a03	Beegfs_M9_M10_M13_M14	答
meta_14.yml	8065	i2b：100.128.102.14/16 i1b：100.127.101.14/16	0	ictad22a03	Beegfs_M9_M10_M13_M14	B
meta_15.yml	8075	i3b：100.127.101.15或16 i4b：100.128.102.15或16	1.	ictad22a04	Beegfs_M11_M12_M15_16	答
meta_16.yml	8085	i4b：100.128.102.16/16 i3b：100.127.101.16/16	1.	ictad22a04	Beegfs_M11_M12_M15_16	B

3. 在`group_vars/下、`使用以下模板为资源组`stor_09`到`stor_16`创建文件、然后参考以下示例填写每个服务的占位值：

```
# stor_0X - BeeGFS HA Storage Resource Group
beegfs_ha_beegfs_storage_conf_resource_group_options:
  connStoragePortTCP: <PORT>
  connStoragePortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: storage
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid6
        criteria_drive_count: 10
        common_volume_configuration:
          segment_size_kb: 512        volumes:
          - size: 21.50 # See note below!
            owning_controller: <OWNING CONTROLLER>
          - size: 21.50            owning_controller: <OWNING
CONTROLLER>
```

有关要使用的正确大小、请参见 "建议的存储池过度配置百分比"...

文件名	Port	浮动IP	NUMA区域	块节点	存储池	所属控制器
STOR_09.yml	8013	i1b：100.127.103.9/16 i2b：100.128.104.9/16	0	ictad22a03	beegfs_s9_s10	答
STOR_10.yml	8023	i2b：100.128.104.10/16 i1b：100.127.103.10/16	0	ictad22a03	beegfs_s9_s10	B
STOR_11.yml	8033	i3b：100.127.103.11/16 i4b：100.128.104.11/16	1.	ictad22a04	beegfs_s11_s12	答
STOR_12.yml	8043	i4b：100.128.104.12/16 i3b：100.127.103.12/16	1.	ictad22a04	beegfs_s11_s12	B

文件名	**Port**	浮动**IP**	**NUMA**区域	块节点	存储池	所属控制器
STOR_13.yml	8053	i1b ：100.127.103.13/16 i2b ：100.128.104.13/ 16	0	ictad22a03	beegfs_s13_s14	答
STOR_14.yml	8063	i2b ：100.128.104.14/16 i1b ：100.127.103.14/16	0	ictad22a03	beegfs_s13_s14	B
STOR_15.yml	8073.	i3b ：100.127.103.15/16 i4b ：100.128.104.15/16	1.	ictad22a04	beegfs_s15_s16	答
STOR_16.yml	8083.	i4b ：100.128.104.16/16 i3b ：100.127.103.16/16	1.	ictad22a04	beegfs_s15_s16	B

第**4**步：为纯存储组件配置清单

以下步骤介绍如何为BeeGFS纯存储组件设置Ansible清单。设置元数据+存储与仅存储组件的配置之间的主要区别是、省略了所有元数据资源组、并将每个存储池的`Criteria_drive_count`从10更改为12。

步骤

1. 在`inventory.yml`中、在现有配置下填充以下参数：

```
# ictad22h05/ictad22h06 HA Pair (storage only building block):
  stor_17:
    hosts:
      ictad22h05:
      ictad22h06:
  stor_18:
    hosts:
      ictad22h05:
      ictad22h06:
  stor_19:
    hosts:
      ictad22h05:
      ictad22h06:
  stor_20:
    hosts:
      ictad22h05:
      ictad22h06:
  stor_21:
    hosts:
      ictad22h06:
      ictad22h05:
  stor_22:
    hosts:
      ictad22h06:
      ictad22h05:
  stor_23:
    hosts:
      ictad22h06:
      ictad22h05:
  stor_24:
    hosts:
      ictad22h06:
      ictad22h05:
```

2. 在`group_vars/`下、使用以下模板为资源组`stor_17`到`stor_24`创建文件、然后参考以下示例填写每个服务的占位值：

```
# stor_0X - BeeGFS HA Storage Resource Group
beegfs_ha_beegfs_storage_conf_resource_group_options:
  connStoragePortTCP: <PORT>
  connStoragePortUDP: <PORT>
  tuneBindToNumaZone: <NUMA ZONE>
floating_ips:
  - <PREFERRED PORT:IP/SUBNET>
  - <SECONDARY PORT:IP/SUBNET>
beegfs_service: storage
beegfs_targets:
  <BLOCK NODE>:
    eseries_storage_pool_configuration:
      - name: <STORAGE POOL>
        raid_level: raid6
        criteria_drive_count: 12
        common_volume_configuration:
          segment_size_kb: 512
        volumes:
          - size: 21.50 # See note below!
            owning_controller: <OWNING CONTROLLER>
          - size: 21.50
            owning_controller: <OWNING CONTROLLER>
```

有关要使用的正确大小、请参见 "建议的存储池过度配置百分比"。

文件名	**Port**	浮动**IP**	**NUMA**区域	块节点	存储池	所属控制器
STOR_17.yml	8013	i1b：100.127.103.17/16 i2b：100.128.104.17/16	0	ictad22a05	beegfs_s17_s18	答
STOR_18.yml	8023	i2b：100.128.104.18/16 i1b：100.127.103.18/16	0	ictad22a05	beegfs_s17_s18	B
STOR_19.yml	8033	i3b：100.127.103.19/16 i4b：100.128.104.19/16	1.	ictad22a06	beegfs_s19_s20	答

文件名	Port	浮动IP	NUMA区域	块节点	存储池	所属控制器
STOR_20.yml	8043	i4b：100.128.104.20/16 i3b：100.127.103.20/16	1.	ictad22a06	beegfs_s19_s20	B
STOR_21.yml	8053	i1b：100.127.103.21/16 i2b：100.128.104.21/16	0	ictad22a05	Beegfs_S21_S22	答
STOR_22.yml	8063	i2b：100.128.104.22/16 i1b：100.127.103.22/16	0	ictad22a05	Beegfs_S21_S22	B
STOR_23.yml	8073.	i3b：100.127.103.23/16 i4b：100.128.104.23/16	1.	ictad22a06	beegfs_s23_s24	答
STOR_24.yml	8083.	i4b：100.128.104.24/16 i3b：100.127.103.24/16	1.	ictad22a06	beegfs_s23_s24	B

部署BeeGFS

部署和管理配置涉及运行一个或多个攻略手册、其中包含Ansible需要执行的任务、并将整个系统置于所需状态。

虽然所有任务都可以包含在一本攻略手册中、但对于复杂的系统、这很快就会变得难以管理。Ansible可用于创建和分发角色、以便打包可重复使用的攻略手册和相关内容(例如：默认变量、任务和处理程序)。有关详细信息、请参见的Ansible文档 "角色"。

角色通常作为包含相关角色和模块的Ansible集合的一部分进行分布。因此、这些攻略手册主要仅导入分布在各种NetApp E系列Ansible资料集中的多个角色。

> 目前、部署BeeGFS至少需要两个组件(四个文件节点)、除非将单独的仲裁设备配置为Tiebreaker、以缓解在双节点集群中建立仲裁时出现的任何问题。

步骤

1. 创建新的`playbook.yml`文件并包括以下内容：

```
# BeeGFS HA (High Availability) cluster playbook.
- hosts: eseries_storage_systems
```

```
  gather_facts: false
  collections:
    - netapp_eseries.santricity
  tasks:
    - name: Configure NetApp E-Series block nodes.
      import_role:
        name: nar_santricity_management
- hosts: all
  any_errors_fatal: true
  gather_facts: false
  collections:
    - netapp_eseries.beegfs
  pre_tasks:
    - name: Ensure a supported version of Python is available on all
file nodes.
      block:
        - name: Check if python is installed.
          failed_when: false
          changed_when: false
          raw: python --version
          register: python_version
        - name: Check if python3 is installed.
          raw: python3 --version
          failed_when: false
          changed_when: false
          register: python3_version
          when: 'python_version["rc"] != 0 or (python_version["stdout"]
| regex_replace("Python ", "")) is not version("3.0", ">=")'
        - name: Install python3 if needed.
          raw: |
            id=$(grep "^ID=" /etc/*release* | cut -d= -f 2 | tr -d '"')
            case $id in
              ubuntu) sudo apt install python3 ;;
              rhel|centos) sudo yum -y install python3 ;;
              sles) sudo zypper install python3 ;;
            esac
          args:
            executable: /bin/bash
          register: python3_install
          when: python_version['rc'] != 0 and python3_version['rc'] != 0
          become: true
        - name: Create a symbolic link to python from python3.
          raw: ln -s /usr/bin/python3 /usr/bin/python
          become: true
          when: python_version['rc'] != 0
      when: inventory_hostname not in
```

```
groups[beegfs_ha_ansible_storage_group]
    - name: Verify any provided tags are supported.
      fail:
        msg: "{{ item }} tag is not a supported BeeGFS HA tag. Rerun
your playbook command with --list-tags to see all valid playbook tags."
      when: 'item not in ["all", "storage", "beegfs_ha",
"beegfs_ha_package", "beegfs_ha_configure",
"beegfs_ha_configure_resource", "beegfs_ha_performance_tuning",
"beegfs_ha_backup", "beegfs_ha_client"]'
      loop: "{{ ansible_run_tags }}"
  tasks:
    - name: Verify before proceeding.
      pause:
        prompt: "Are you ready to proceed with running the BeeGFS HA
role? Depending on the size of the deployment and network performance
between the Ansible control node and BeeGFS file and block nodes this
can take awhile (10+ minutes) to complete."
    - name: Verify the BeeGFS HA cluster is properly deployed.
      import_role:
        name: beegfs_ha_7_2
```

> 本攻略手册将运行几个`pre_tasks`、用于验证Python 3是否已安装在文件节点上、并检查所提供的Ansible标记是否受支持。

2. 准备好部署BeeGFS时、请使用`Ansible攻略手册`命令和清单和攻略手册文件。

   此部署将运行所有`pre_tasks`、然后提示用户确认、然后再继续实际的BeeGFS部署。

   运行以下命令、根据需要调整货叉数量(请参见以下注释)：

```
ansible-playbook -i inventory.yml playbook.yml --forks 20
```

> 特别是对于大型部署、建议使用`fork`参数覆盖默认的货叉数(5)、以增加Ansible并行配置的主机数。(有关详细信息、请参见 "Ansible性能调整" 和 "控制攻略手册执行") 最大值设置取决于Ansible控制节点上可用的处理能力。以上示例20是在具有4个CPU (Intel ® Xeon ® Gold 6146 CPU @ 3.20GHz)的虚拟Ansible控制节点上运行的。

根据Ansible控制节点与BeeGFS文件和块节点之间的部署规模和网络性能、部署时间可能会有所不同。

配置BeeGFS客户端

您必须在需要访问BeeGFS文件系统的任何主机上安装和配置BeeGFS客户端、例如计算或GPU节点。在此任务中、您可以使用Ansible和BeeGFS收集。

步骤

1. 如果需要、请从Ansible控制节点向要配置为BeeGFS客户端的每个主机设置无密码SSH：

```
ssh-copy-id <user>@<HOSTNAME_OR_IP>
```

2. 在`host_vars/下、为名为<HOSTNAME>.yml`的每个BeeGFS客户端创建一个包含以下内容的文件、并使用适用于您环境的正确信息填写占位符文本：

```
# BeeGFS Client
ansible_host: <MANAGEMENT_IP>
# OPTIONAL: If you want to use the NetApp E-Series Host Collection's
IPoIB role to configure InfiniBand interfaces for clients to connect to
BeeGFS file systems:
eseries_ipoib_interfaces:
  - name: <INTERFACE>
    address: <IP>/<SUBNET_MASK> # Example: 100.127.1. 1/16
  - name: <INTERFACE>0
    address: <IP>/<SUBNET_MASK>
```

> 目前、必须在每个客户端上配置两个InfiniBand接口、两个存储IPoIB子网中每个接口一个。如果对此处列出的每个BeeGFS服务使用示例子网和建议范围、则客户端应配置一个介于`100.127.1范围内的接口。0`到`100.127.99.255`、另一个位于`100.128.1中。0`到`100.128。99.255`。

3. 创建一个新文件`client_inventory.yml`、然后在顶部填充以下参数：

```
# BeeGFS client inventory.
all:
  vars:
    ansible_ssh_user: <USER> # This is the user Ansible should use to
connect to each client.
    ansible_become_password: <PASSWORD> # This is the password Ansible
will use for privilege escalation, and requires the ansible_ssh_user be
root, or have sudo privileges.
The defaults set by the BeeGFS HA role are based on the testing
performed as part of this NetApp Verified Architecture and differ from
the typical BeeGFS client defaults.
```

> 请勿以纯文本格式存储密码。请改用Ansible Vault (请参见的Ansible文档) "使用Ansible Vault加密内容")或在运行攻略手册时使用`-ask-pass-pass`选项。

4. 在`client_inventory.yml`文件中、列出应在`beegfs_clients`组下配置为BeeGFS客户端的所有主机、然后指定构建BeeGFS客户端内核模块所需的任何其他配置。

```
  children:
    # Ansible group representing all BeeGFS clients:
    beegfs_clients:
      hosts:
        ictad21h01:
        ictad21h02:
        ictad21h03:
        ictad21h04:
        ictad21h05:
        ictad21h06:
        ictad21h07:
        ictad21h08:
        ictad21h09:
        ictad21h10:
      vars:
        # OPTION 1: If you're using the Mellanox OFED drivers and they
are already installed:
        eseries_ib_skip: True # Skip installing inbox drivers when using
the IPoIB role.
        beegfs_client_ofed_enable: True
        beegfs_client_ofed_include_path:
"/usr/src/ofa_kernel/default/include"
        # OPTION 2: If you're using inbox IB/RDMA drivers and they are
already installed:
        eseries_ib_skip: True # Skip installing inbox drivers when using
the IPoIB role.
        # OPTION 3: If you want to use inbox IB/RDMA drivers and need
them installed/configured.
        eseries_ib_skip: False # Default value.
        beegfs_client_ofed_enable: False # Default value.
```

使用Mellanox OFED驱动程序时、请确保`beegfs_client_OFED_include_path`指向适用于Linux安装的正确"header include path"。有关详细信息、请参见的BeeGFS文档 "RDMA支持"。

5. 在`client_inventory.yml`文件中、列出要挂载到任何先前定义的`vars`底部的BeeGFS文件系统。

```
        beegfs_client_mounts:
          - sysMgmtdHost: 100.127.101.0 # Primary IP of the BeeGFS
management service.
            mount_point: /mnt/beegfs    # Path to mount BeeGFS on the
client.
            connInterfaces:
              - <INTERFACE> # Example: ibs4f1
              - <INTERFACE>
            beegfs_client_config:
              # Maximum number of simultaneous connections to the same
node.
              connMaxInternodeNum: 128 # BeeGFS Client Default: 12
              # Allocates the number of buffers for transferring IO.
              connRDMABufNum: 36 # BeeGFS Client Default: 70
              # Size of each allocated RDMA buffer
              connRDMABufSize: 65536 # BeeGFS Client Default: 8192
              # Required when using the BeeGFS client with the shared-
disk HA solution.
              # This does require BeeGFS targets be mounted in the
default "sync" mode.
              # See the documentation included with the BeeGFS client
role for full details.
              sysSessionChecksEnabled: false
```

`beegfs_client_config`表示已测试的设置。有关所有选项的全面概述、请参见`netapp_eseries.beegfs`集合的`beegfs_client` role附带的文档。其中包括有关挂载多个BeeGFS文件系统或多次挂载相同BeeGFS文件系统的详细信息。

6. 创建新的`client_playbook.yml`文件、然后填充以下参数：

```
# BeeGFS client playbook.
- hosts: beegfs_clients
  any_errors_fatal: true
  gather_facts: true
  collections:
    - netapp_eseries.beegfs
    - netapp_eseries.host
  tasks:
    - name: Ensure IPoIB is configured
      import_role:
        name: ipoib
    - name: Verify the BeeGFS clients are configured.
      import_role:
        name: beegfs_client
```

> 如果您已在相应的IPoIB接口上安装了所需的IB/RDMA驱动程序和配置的IP、请省略导入`NetApp_E系列.host`集合和`IPoIB`角色。

7. 要安装和构建客户端并挂载BeeGFS、请运行以下命令：

```
ansible-playbook -i client_inventory.yml client_playbook.yml
```

8. 在将BeeGFS文件系统投入生产之前、我们*强烈*建议您登录到任何客户端并运行`beegfs-fsck -checkfs`以确保所有节点均可访问且未报告任何问题。

扩展到五个组件以上

您可以将Pacemaker和Corosync配置为扩展到五个组件(10个文件节点)以上。但是、较大的集群存在一些缺点、最终、Pacemaker和Corosync最多会占用32个节点。

NetApp仅测试了最多10个节点的BeeGFS HA集群；不建议或不支持将单个集群扩展到超过此限制的范围。但是、BeeGFS文件系统仍需要扩展到10个以上的节点、而NetApp已在NetApp解决方案 上的BeeGFS中对此进行了说明。

通过在每个文件系统中部署包含一个子块的多个HA集群、您可以扩展整个BeeGFS文件系统、而不受底层HA集群机制的任何建议限制或硬限制。在此情景中、请执行以下操作：

- 创建一个新的Ansible清单以表示其他HA集群、然后省略配置另一个管理服务。请将每个附加集群`ha_cluster.yml`中的`beegfs_ha_mgmtd_floating_ip`变量指向第一个BeeGFS管理服务的IP。
- 在向同一文件系统添加其他HA集群时、请确保满足以下要求：
  - BeeGFS节点ID是唯一的。
  - 与`group_vars`下的每个服务对应的文件名在所有集群中都是唯一的。
  - BeeGFS客户端和服务器IP地址在所有集群中都是唯一的。
  - 在尝试部署或更新其他集群之前、包含BeeGFS管理服务的第一个HA集群正在运行。
- 在每个HA集群各自的目录树中单独维护清单。

  尝试在一个目录树中混用多个集群的清单文件可能会导致发生原因 对BeeGFS HA角色聚合应用于特定集群的配置的方式产生问题。

> 无需在创建新的HA集群之前将每个HA集群扩展到五个组件。在许多情况下、每个集群使用较少的组件会更易于管理。一种方法是将每个机架中的组件配置为HA集群。

建议的存储池过度配置百分比

如果对第二代构建块采用每个存储池配置的标准四个卷、请参见下表。

下表提供了在`eseries_storage_pool_configuration`中为每个BeeGFS元数据或存储目标用作卷大小的建议百分比：

驱动器大小	**Size**
1.92 TB	18
3.84 TB	21.5.
7.68 TB	22.5
15.3 TB	24

> 上述指南不适用于包含管理服务的存储池、该存储池应将上述大小减少0.25%、以便为管理数据分配1%的存储池。

要了解如何确定这些值、请参见 "TR-4800：附录A：了解SSD持久性和过度配置"。

法律声明

法律声明提供对版权声明、商标、专利等的访问。

版权

http://www.netapp.com/us/legal/copyright.aspx

商标

NetApp 、 NetApp 徽标和 NetApp 商标页面上列出的标记是 NetApp 、 Inc. 的商标。其他公司和产品名称可能是其各自所有者的商标。

http://www.netapp.com/us/legal/netapptmlist.aspx

专利

有关 NetApp 拥有的专利的最新列表，请访问：

https://www.netapp.com/us/media/patents-page.pdf

隐私政策

https://www.netapp.com/us/legal/privacypolicy/index.aspx

开放源代码

通知文件提供有关 NetApp 软件中使用的第三方版权和许可证的信息。

"E 系列 /EF 系列 SANtricity 操作系统注意事项"

版权信息

版权所有©2022 NetApp、Inc.。保留所有权利。Printed in the U.S.版权所涵盖的本文档的任何部分不得以任何形式或任何手段复制、包括影印、录制、 磁带或存储在电子检索系统中—未经版权所有者事先书面许可。

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

本软件由NetApp按"原样"提供、不含任何明示或默示担保、包括但不限于适销性和特定用途适用性的默示担保、特此声明不承担任何责任。IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice.NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp.The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S.patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

商标信息

NetApp、NetApp标识和中列出的标记 http://www.netapp.com/TM 是NetApp、Inc.的商标。其他公司和产品名称可能是其各自所有者的商标。